

The background of the entire image is a deep space photograph. The top half features a dark blue and black sky filled with numerous small, bright stars. A faint, wispy nebula is visible in the upper center. The middle section is a solid teal band where the text is located. The bottom half shows a more dramatic scene with a large, glowing orange and yellow nebula on the left, and a bright star with a prominent diffraction pattern in the lower center.

Cloud Computing

Il bello di vivere

Vincenzo Salvatore

Raffaele Mercurio



Contents

1	Introduction	5
1.1	Essential Characteristics of Cloud Computing (NIST)	5
1.2	Service Models	6
1.3	Deployment Models	7
1.4	Evaluation of Cloud Computing Services - NIST 800-145	8
1.5	The Cloud computing Reference Model - NIST SP 500-292	8
1.5.1	Example of Cloud Roles and their interactions	9
1.6	Service Orchestration	10
1.7	Cloud Service Management	11
2	Enabling technologies	13
2.1	Distributed Systems	13
2.1.1	Distributed Computing	13
2.2	Components of a Distributed System	14
2.2.1	Architectural Style	14
2.2.2	Roles of components	15
2.3	Service Orientation	16
2.3.1	Charateristics of Services	16
2.3.2	Major Roles	16
2.3.3	Benefits of SOA	16

2.4	Orchestration and Choreography	17
2.5	Microservices	17
2.5.1	Scalability	17
2.5.2	Monolith Features	17
2.5.3	Monolith vs. Microservices	18
2.5.4	SOA vs. Microservices	18
3	Containerization	21
3.1	Containerization	21
3.1.1	Foundations of Containerization	21
3.2	Docker	22
3.2.1	Storage Options	22
3.2.2	Networking	23
4	Virtualization	25
4.1	Virtualization	25
4.1.1	Main Features	25
4.1.2	Layered Structure	25
4.1.3	Privilege of Instructions	25
4.2	Virtual Machine Manager and Hypervisor	26
4.2.1	Level of Virtualization	26
4.2.2	Properties to Satisfy	26
4.2.3	Composition	27
4.3	Full Virtualization	27
4.3.1	Problem of Virtualization in x86 OS	27
4.4	Paravirtualization	28
5	Automation & Orchestration	29
5.1	Automation vs Autonomic Computing	29
5.1.1	Data Centers Automation	29
5.2	Orchestration	32
5.3	Container Orchestration	32
5.3.1	Kubernetes	32

6	Cloud Storage	37
6.1	Basic Concepts	37
6.1.1	Atomicity	37
6.1.2	Storage Model	38
6.1.3	Consensus	38
6.2	Google File System (GFS)	39
6.3	Hadoop Distributed File System (HDFS)	39
6.4	Amazon Simple Storage Service (S3)	40
6.4.1	RW failures	40
6.4.2	Data Consistency Model	40
6.5	NoSQL	40
6.5.1	Databases	40
6.5.2	BigTable	41
6.5.3	Amazon DynamoDB	42
7	FC1 - Definition and Architecture	45
7.1	Task 1	45
7.1.1	Use Case 1: Video Conference System	45
7.1.2	Answers Use Case 1	46
7.1.3	Use Case 2: Scientific Computing Platform	47
7.1.4	Answers Use Case 2	48
7.2	Task 2	48
7.2.1	Deconstructing the prerequisites	48
7.2.2	The Matching Criterion	49
7.2.3	Metrics of Speed vs. Precision	49
7.2.4	Elasticity vs. Efficiency	50
7.2.5	The Impact of Discrete Scaling Units	50
7.2.6	Benchmarking Real-Life Scenarios	51
8	FC2 - Autonomic Computing	53
8.1	Task 1 - Foundations and Inspirations	53
8.1.1	The Biological Metaphor	53
8.1.2	Homeostasis vs. Vital Parameters	53
8.1.3	The Role of Motility	54
8.1.4	Scientific Cross-Pollination	54
8.1.5	Microeconomics in Resource Allocation	54
8.1.6	Nash Equilibrium vs. Pareto Optimality	55
8.1.7	Stackelberg vs. Conjectural Equilibrium	55

8.1.8	Complex Adaptive Systems (CAS)	55
8.1.9	Emergence and Self-Organisation	56
8.1.10	Ashby's Ultra-Stable System	56
8.2	Task 2 - Autonomic Architectures	56
8.2.1	The MAPE-K Loop Anatomy	56
8.2.2	Touchpoints: Sensors and Effectors	57
8.2.3	Innate vs. Acquired Knowledge	57
8.2.4	Rule-Based (Reflex) Reasoning	57
8.2.5	Model-Driven Autonomicity	58
8.2.6	Goal-Based vs. Utility-Based Systems	58
8.2.7	Search-Based Reasoning	58
8.2.8	Architectural Models and Runtime Reflectivity	59
8.2.9	Probabilistic Reasoning and Uncertainty	59
8.2.10	Autonomic Systems That Learn	59
8.2.11	Self-management In Practice	59
9	FC3 - Autoscaling	61
9.1	Task 1	61
10	FC4 - Cloud Storage: GFS & HDFS	63
10.1	Task 1	63
11	FC5 - Cloud DB: Dynamo & BigTable	65
11.1	Task 1	65



1. Introduction

Cloud computing is essentially the evolution of computing into a public utility, much like electricity, water, or the telephone system. It is a model that allows consumers to access computing services on demand and pay only for what they use.

Key Word

Ubiquitous: server and data can be located anywhere and accessed from anywhere

This **cloud model** is composed of **five** essential characteristics, **three** service models, and **four** deployment models

1.1 Essential Characteristics of Cloud Computing (NIST)

1. **On-demand self-service:** users can automatically provision computing resources without human intervention from the service provider.
 - **Option A:** Fully automated service provisioning
 - **Option B:** The provider use an automated interface to request and truck the service, but provider may use manual labor to provisioning the interface.
2. **Broad network access:** Possibility to access cloud resourcess from internet by using different devices.
 - **Option A:** Available over the internet
 - **Option B:** Available over a network that is available from all the access points the customer requires.

3. **Resource pooling:** Computing resources are pooled to serve multiple consumer using a **Multi-tenant model**.

Multi-Tenant Model

The computing infrastructure is shared among more than one customer, following this principles:

- multi users (tenants) access the same application logic simultaneously.
 - Each tenant has its own view of the application as a dedicated instance.
 - Multitenant application isolate data and configuration for each tenant.
4. **Rapid elasticity:** Capability of the system to adapt to workload that rapidly changes.
 - **Option A:** Resource allocation is automated and near to real time.
 - **Option B:** Not fully automated, but fast enough to support the requirements.

Two type of Scaling:

- **Vertical Scaling:** increase the capacity of a single resource (e.g., adding more CPU or RAM to a server).
- **Horizontal Scaling :** add more instances of a resource (e.g., adding more servers to a cluster).

Example

Black Friday is a prime example of a situation where rapid elasticity is crucial. During this shopping event, there is a significant surge in online traffic and demand for products. E-commerce platforms need to quickly scale their resources to handle the increased load and ensure a smooth shopping experience for customers. Cloud computing allows these platforms to rapidly allocate additional resources, such as servers and bandwidth, to accommodate the spike in traffic without any downtime or performance issues.

5. **Measured service:** Cloud service characteristics are strictly monitored with enough details in order to guarantee specific **Service Level Agreement (SLA)** and then charge customers for that services.

1.2 Service Models

The **service models** in cloud computing are designed to define the level of control a *consumer* has over the cloud resources and to clearly separate the responsibilities between the **cloud consumer (subscriber)** and the **cloud provider**. According to the *NIST framework*[ahmad2023deep], there are three primary service models:

1. **Infrastructure as a Service (IaaS):** The consumer has control over operating systems, storage, and deployed applications; and possibly limited control of select networking component.

Example

Examples include Amazon Web Services (AWS) EC2, Microsoft Azure Virtual Machines, and Google Cloud Compute Engine.

2. **Platform as a Service (PaaS):** The consumer has control over the deployed applications and possibly configuration settings for the application-hosting environment.

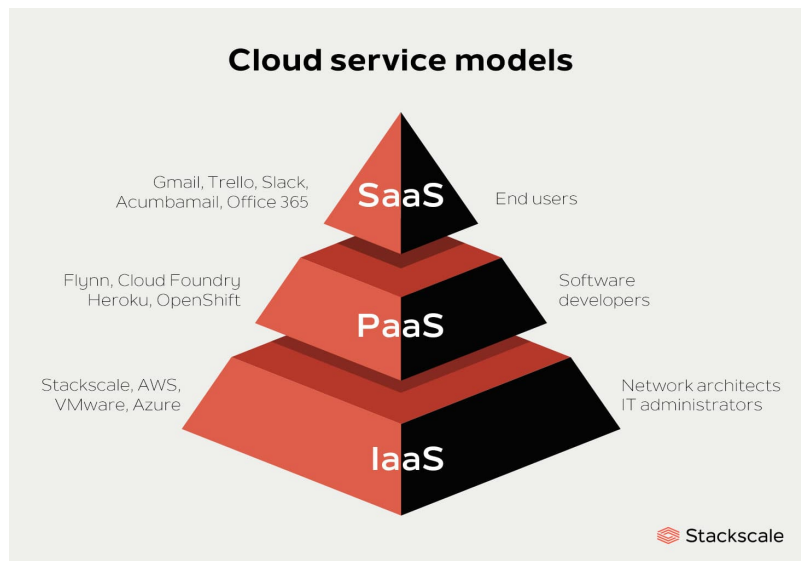
Example

Examples include Google App Engine, Microsoft Azure App Service, and Heroku

3. **Software as a Service (SaaS):** The cloud provider hosts applications and makes them available to consumers over the internet. Consumers can access and use the software without managing the underlying infrastructure or platform.

Example

Examples include Google Workspace (formerly G Suite), Microsoft Office 365, and Salesforce.



1.3 Deployment Models

While service models determine what you get, *deployment models* determine *how the infrastructure is hosted, who manages it, and who is allowed to access it*. According to the *NIST framework*[ahmad2023deep], there are four primary deployment models:

1. **Public Cloud:** This is the most open deployment model. The cloud infrastructure is provisioned for open use by the general public and is owned, managed, and operated by a cloud provider. As your notes enthusiastically summarize, it is *All you can access!!!!*
2. **Private Cloud:** Is dedicated entirely to a specific customer, ensuring logical and physical isolation from other users. This model offers maximum control and security.

Note

A closely related concept is the **Virtual Private Cloud (VPC)**. Solutions like Amazon VPC and Google Cloud VPC provide VPN features to securely connect your resources within a public cloud environment while providers like IBM and HP offer cloud environments with enhanced physical isolation.

3. **Hybrid Cloud:** This model is a combination of two or more distinct cloud infrastructures, such as mixing a private cloud with a public cloud.
4. **Community Cloud:** Listed as the fourth model by NIST, this is a cloud infrastructure shared by several organizations that support a specific community with shared concerns.

Example

Examples include government agencies sharing a cloud for public services or research institutions collaborating on a shared cloud for scientific research.

1.4 Evaluation of Cloud Computing Services - NIST 800-145

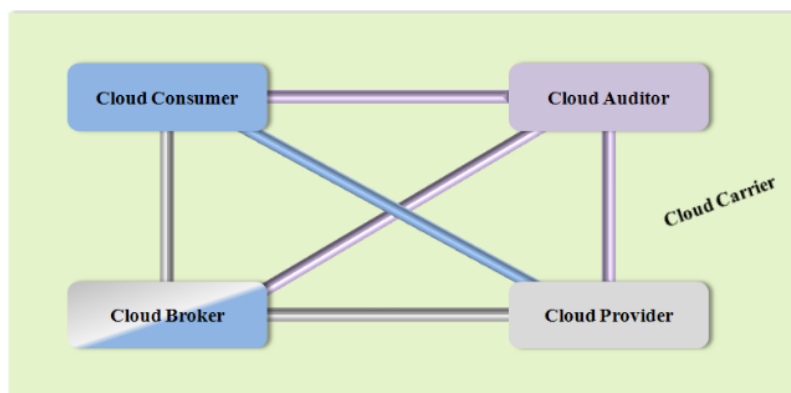
This section is used to determine whether a specific capability is actually a cloud service, allowing you to verify it through three specific tools:

1. **Cloud Service Worksheet:** to confirm if a service is a cloud service, just looking **Essential Characteristics of Cloud Computing??**.
2. **Cloud Service Model Worksheet:** to determine the service model of a cloud service, just looking **Service Models**1.2.
3. **Cloud Deployment Model Worksheet:** to determine the deployment model of a cloud service, just looking **Deployment Models**1.3.

1.5 The Cloud computing Reference Model - NIST SP 500-292

In this section, we will focus on **Cloud Roles**, the framework defines five major actors that interact within this cloud ecosystem:

1. **Cloud Consumer:** The individual or organization that uses the cloud services.
2. **Cloud Provider:** The entity responsible for making the cloud services available to consumers.
3. **Cloud Auditor:** An independent party that assesses the security and performance of the cloud services.
4. **Cloud Broker:** An intermediary that manages the use, performance, and delivery of cloud services, and negotiates relationships between cloud providers and consumers.
5. **Cloud Carrier:** The intermediary that provides connectivity and transport of cloud services from providers to consumers.



1.5.1 Example of Cloud Roles and their interactions

Cloud Broker



The **Cloud Broker** acts as a middleman between the consumer and various cloud providers. Instead of the consumer buying and managing services directly from multiple companies, the broker aggregates, integrates, or customizes these services into a single, cohesive package. It can provide services in three categories:

1. **Service Intermediation:** A Cloud Broker can enhance a service by adding value to it, such as providing additional security features, performance monitoring, or billing services.
2. **Service Aggregation:** A Cloud Broker can combine services from different providers into a unified offering, simplifying management for the consumer.
3. **Service Arbitrage:** A Cloud Broker can dynamically select services from different providers based on factors like cost, performance, or availability, optimizing the consumer's experience.

Example

Real-world example: Imagine a company that needs storage from Google Cloud, computing power from Amazon Web Services (AWS), and a database from Microsoft Azure. Instead of managing three separate accounts and bills, they hire a Cloud Broker (like Flexera or a specialized IT consultancy). The broker provides a single dashboard to manage everything, optimizing costs and handling the complex technical integration behind the scenes.

Cloud Auditor

The **Cloud Auditor** is an independent, third-party organization. Its job is to objectively evaluate the cloud provider's services, focusing heavily on security protocols, privacy standards, and performance metrics. This ensures the provider is actually delivering what they promised and adhering to legal regulations.



Example

Real-world example: A bank wants to move its sensitive financial data to a new Cloud Provider but needs to be absolutely certain the data won't be compromised. An independent Cloud Auditor (like Deloitte or a certified ISO 27001 inspector) runs a comprehensive security check on the provider's servers. They then issue an official certification report. The bank (Consumer) reviews this audit report to verify the Provider's reliability before signing the final contract.

1.6 Service Orchestration

In the NIST Reference Architecture, **Service Orchestration** is a core responsibility of the **Cloud Provider**. It refers to the composition, arrangement, coordination, and management of various system components and computing resources in order to successfully deliver cloud services to the **Cloud Consumer**. To make this work, the architecture uses a three-layered model that directly integrates the concepts we have discussed so far:

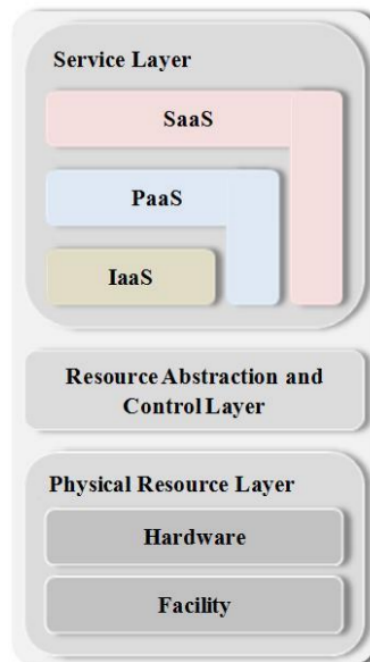


Figure 1.1: Service Orchestration in the NIST Reference Architecture

1. **Service Layer:** This is the interface layer where the Cloud Provider actually exposes the services to the Cloud Consumer. Depending on the model the consumer chooses, the orchestration tasks differ.

Example

For example:

- **IaaS**: Requires the orchestration of Virtual Machines (VMs) and virtual devices;
- **PaaS**: Often relies on container orchestration (like Kubernetes);
- **SaaS**: Relies on orchestrating microservices and Service-Oriented Architecture (SOA)

2. **Resource Abstraction Layer**: This is the software fabric that bridges the gap between the physical hardware and the services you buy. It uses software components like hypervisors to create virtual machines, virtual storage, and network abstractions.

Note

This layer is what makes the **Five Essential Characteristics** possible. The control software in this layer handles resource allocation and usage monitoring, which directly enables Resource Pooling, Rapid Elasticity, and Measured Service.

3. **Physical Resource Layer**: This foundational layer consists of all the actual physical equipment. It includes the hardware (CPUs, memory, network routers, firewalls, switches, and hard disks).

1.7 Cloud Service Management

Cloud Service Management is a core architectural component managed by the Cloud Provider that encompasses all the service-related functions necessary for the management and operation of the services required by, or proposed to, cloud consumers.

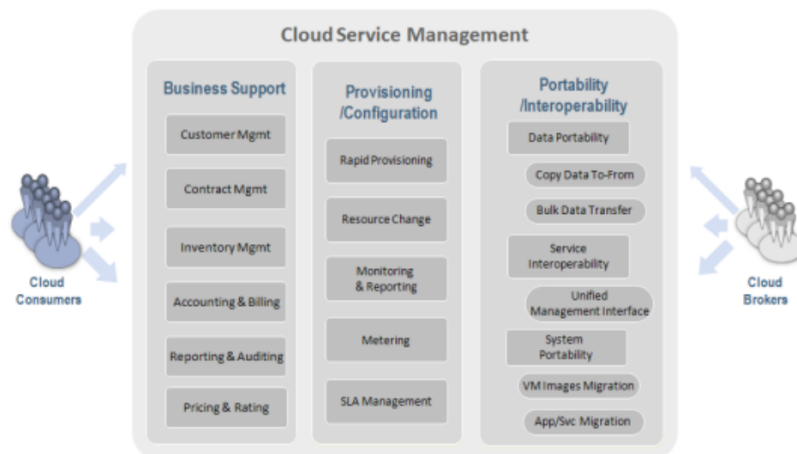


Figure 1.2: Cloud Service Management in the NIST Reference Architecture

According to the NIST Reference Architecture, is divided into three main operational perspectives:

1. **Business Support**: This area entails the set of business-related services dealing with client-facing operations and supporting processes.

2. **Provisioning and Configuration:** This category focuses on the technical delivery, monitoring, and lifecycle management of the cloud resources
3. **Portability and Interoperability:** Are crucial capabilities designed to prevent vendor lock-in (remaining prisoner of only one cloud provider), and give consumers flexibility across multiple cloud environments.
 - **Portability:** The ability to move applications and data from one cloud provider to another without significant rework.
 - **Interoperability:** The ability of different cloud services and platforms to work together seamlessly, allowing for integration and communication across different cloud environments.



2. Enabling technologies

2.1 Distributed Systems

A **distributed system** is a collection of independent computers that appears to its users as a single coherent system.

2.1.1 Distributed Computing

Distributed computing, differently from the concept of distributed systems, is a **paradigm**.

Define **How the computation is carried on**. In a distributed architecture, a massive or complex task is not handled sequentially by a single machine. Instead, the computation is divided into smaller, manageable units that are executed concurrently (at the exact same time) This parallel execution is what allows distributed systems to process massive amounts of data much faster than a single computer ever could.

The computing elements

These concurrent units of computation can be assigned to various types of hardware setups

- Cores within the same processor
- Processors on the same computer
- Processors on different nodes

Different Locations

When we say a system is "distributed", it often implies that the physical computing elements are not located in the same place. They could be in different server racks within a single room, or spread across completely different data centers around the world. This geographical distribution is critical because it ensures **reliability**

and fault tolerance;

if one location loses power, the computation can continue elsewhere.

Heterogeneity (Hardware and Software)

This is one of the most powerful features of distributed computing. The nodes making up the system do not need to be identical clones of one another.

- **Hardware heterogeneity:** You can combine powerful, expensive servers with smaller, cheaper commodity hardware within the same network.
- **Software heterogeneity:** These machines can run entirely different operating systems or programming environments.

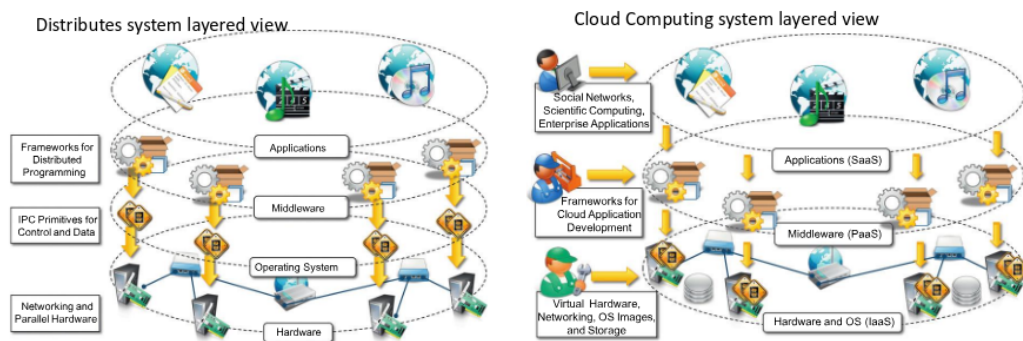


Figure 2.1: Cloud computing is a specialized form of distributed computing that introduces service model.

2.2 Components of a Distributed System

The components of a distributed system are:

- **Applications;**
- **Middleware:** is the part involved in the management of the lower layers in order to make look like a unique system;
- **Operating Systems;**
- **Hardware.**

Cloud Computing is a specialized form of distributed system, where:

- Application $\equiv$ SaaS
- Middleware $\equiv$ PaaS
- OS and Hardware $\equiv$ IaaS

2.2.1 Architectural Style

An Architectural style defines the following concepts:

- the different roles of the components in the distributed system;
- how components are distributed across multiple machines;
- what communication mechanisms are used.

2.2.2 Roles of components

Components have their own life-cycle and interact each other to perform their activities. Each process provides services and can use services exposed by other processes. They are physical organized in architectures like client/server, peer-to-peer and service-oriented.

Event-Based Systems

In event-based systems each component publish a collection of events that the others components can subscribe to receive a callback when the event occurs.

Client-Server Architecture Style

In client/server architecture style there is a client that sends a request and a server that elaborates and replies to it. This style is suitable for many-to-one scenarios.

There are three main components (presentation, application logic and data storage) and two possible models:

- **Thin-client model:** client has only the presentation and so the server does all the works (application logic and data storage);
- **Fat-client model:** client does also some application logic.

These three components can be seen as conceptual layers, called **tiers**:

- **Two-Tiers** (classic model): it's suitable only for systems of limited size since it suffers from scalability issues;
- **Three-Tiers** or **N-Tiers**: are more scalable because it's possible distribute the tiers into several computing units.

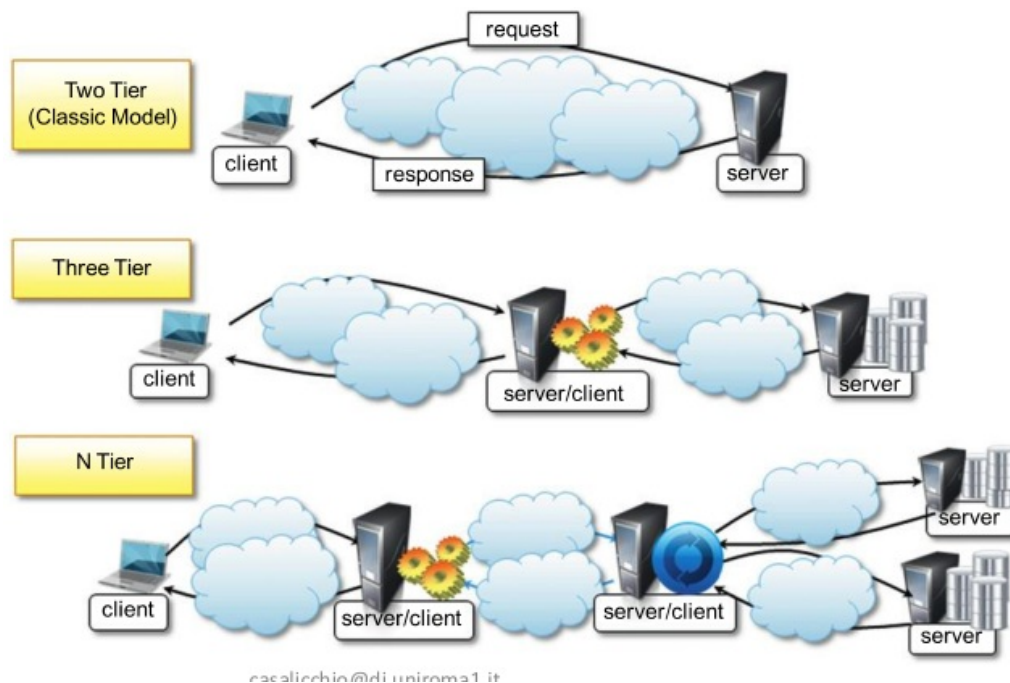


Figure 2.2: Difference between models with two or more tiers.

Peer-to-Peer

In the peer-to-peer architecture style there is symmetry since all the components play the same role and incorporate both client and server capabilities.

It's suitable for highly decentralized systems, like file sharing applications, distributed and cloud storage systems.

Service Oriented Architectures (SOA)

The Service Oriented Architecture is an architectural style that supports service orientation.

2.3 Service Orientation

A service:

- is a logical representation of a repeatable activity that has a specified outcome (like provisioning of weather data or checking customer credit);
- is self-contained (autonomous);
- may be composed of other services;
- seems as a black box to consumers of the service.

2.3.1 Characteristics of Services

An application is an aggregation of services that are coordinated within a SOA. Services have four main characteristics:

- **Explicit Boundaries:** interfaces are kept minimal in order to make simpler reusing them;
- **Autonomy:** services can be integrated in different systems at the same time and can handle failures;
- **Share of schema and contracts:** a contract describes the structure and the order of messages the services can receive;
- **Compatibility based on policy:** the policies describe the requirements of the services.

2.3.2 Major Roles

There are three major roles within SOA:

- **Service Provider:** creates the service and publishes it on a registry;
- **Service Consumer:** locates the service in the registry and use it;
- **Service Registry:** store the service descriptors.

The iter is:

1. the consumer discovers a service on the registry;
2. the registry returns binding informations;
3. the consumer establishes a connection with the provider using the binding informations;
4. the consumer can use the service.

2.3.3 Benefits of SOA

The most relevant benefits of a service-oriented architecture are:

- service re-use;
- message monitoring, control and transformation;

- complex event processing;
- service composition;
- service discovery.

2.4 Orchestration and Choreography

In order to coordinate and manage services, we can use:

- **Service Orchestration:** it coordinates the composition and interaction of services in a centralized way. The orchestrator is responsible for involving and combining the services.

Example

An example of orchestration is the travel planner where a centralized unit reuses all the services, like select destination, display hotels, select hotels, ecc...

- **Service Choreography:** it coordinates the interaction of services without a single point of control. There are some rule of interactions and the coordination happens through the exchange of messages between two or more services.

Example

An example of choreography is the authentication of a system where the client service sends a request to the Auth service, which will check the credentials and will send a request for three factor authentication and so on.

2.5 Microservices

A microservice architecture is the style that structures an application as a collection of services that are:

- highly maintainable and testable;
- independently deployable;
- owned by a small team;
- organized around business capabilities;
- loosely coupled.

This architecture enables the rapid and frequent release of complex applications.

2.5.1 Scalability

Microservices create a new dimension of scalability:

- **X-axis** scaling (horizontal duplication): from one instance to more instances;
- **Y-axis** scaling (functional decomposition): from a monolith to microservices;
- **Z-axis** scaling (data partitioning): from one partition to many partitions;

2.5.2 Monolith Features

A monolithic architecture has some pros:

- it's simple to develop and to drastically change;
- it's easy to scale horizontally (duplication)

But it has also some contra:

- when the complexity increase, it's more difficult to scale.

2.5.3 Monolith vs. Microservices

In the microservice architecture there is an API Gateway to route the requests to different microservices, each one with different databases.

Even the production path is different between these two architecture styles:

- in a monolith, a huge team works on a single code base, creating queues and bottlenecks because of complex testing;
- in microservices, small teams work on separate repositories with automated deployment pipelines, making everything faster and more reliable.

2.5.4 SOA vs. Microservices

SOA and microservices differs about various aspects:

- **Taxonomy:** SOA has many abstract layers (business, middleware, enterpris, application and infrastructure) and requires a high level of coordination between different teams to handle a single request. Microservices have only APIs layer and functional/infrastructure services. So a single team manage all the service without any external coordination.

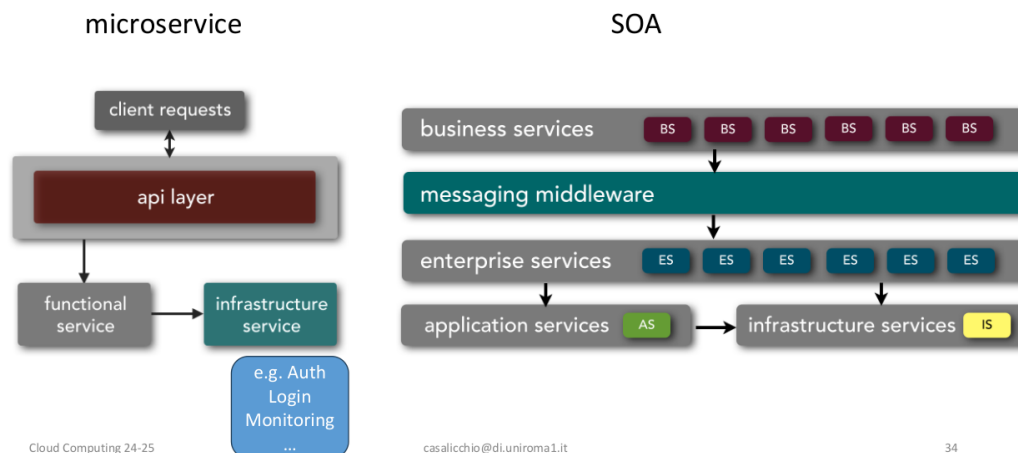


Figure 2.3: Difference of taxonomy between SOA and microservices.

- **Service Ownership and Coordination:** SOA requires coordination while microservices require minimal coordination.
- **Service Granularity:** microservices are single-purpose (specialized) services while services within SOA can range their size from an application service to very large enterprise services.
- **Component Sharing:** SOA follows the share-as-much-as-possible architectural style but this can lead to high connection between components and so there can be more risk associated with changes. Microservices follow the share-as-little-as-possible architectural style, which uses the *bounded context*. This means that all the components and their data represent a closed entity with minimal dependencies.

This style could violate the don't-repeat-yourself principle but it makes maintenance and deployment drastically easier and safer.

- **Service Choreography and Orchestration:** microservices prefer choreography since they don't have a central middleware component. Too much choreography can lead to *effereent coupling*, meaning that if a service depends on different services, the system becomes too rigid.

SOA relies on both orchestrations and choreography.

- **Middleware and API Layer:** SOA relies on messaging middleware to coordinate service calls and translate protocols between incompatible services.

Microservices support the notion of API layer that acts as a service-access interface.

- **Application Scope:** it refers to the overall size of the application that an architecture pattern can support. SOA is well-suited for large and complex systems that require integration with many heterogeneous applications and services.

Microservices architecture is better-suited for smaller, well-partitioned systems.

- **Heterogeneous Interoperability:** SOA accepts every protocol through its middleware and can integrate systems implemented in different programming languages and platforms.

Microservices attempts to simplify the architecture by reducing the number of choices for service integration.



3. Containerization

3.1 Containerization

Containerization is different from virtualization. In the latter we have a hypervisor that manage the communication and the relationship between the system and the VMs that can be hosted. In containerization, instead, we have a unique OS, shared with all the containers.

A container can be used to distribute and run a single application (or a microservice).

Containerization is the evolution of *chroot*, the operation that change the root of the file system for a process in order to isolate it. In this way the process (and its hchildren) cannot have access to other partitions of the file system.

3.1.1 Foundations of Containerization

Containerization is based on:

- **Namespace:** it has the goal of abstract the global system resources (such as PIDs, network devices, mount points, ecc...) in order to give the perception to the processes of being in their own isolated instance of the system.

Changes to the resources in the namespace are visible only by its members and invisible to other processes.

We can create one or more namespaces by creating a process using the `clone()` system call.

We can assign a process to an existing namespace using `setns()`.

We can move a process to a new namespace using `unshare()`.

- **Cgroups:** its main goal is to support resource limiting, prioritization, accounting and controlling. It allows to organize processes into hierarchical groups.

The kernel provides a cgroup interface through a pseudo-filesystem called cgroupfs.

There exists different controller to control different resources (i.e. one for the CPU, one for the memory, one for the PID, ecc...).

- **UnionFS:** it allows to keep files separated physically, but merging them logically into a single view. Each physical directory is called branch and each branch has a different priority. For example if there exist two file with the same name in different branches then in the merge we will have only the one in the branch with largest priority. If there are two directories in different branches but with the same name, in the merge we will see a union of them.

UnionFS allows to mix read-only and read-write branches. In this case the union is read-write, since the copy-on-write mechanism gives the illusion that you can modify files and directories on read-only branches while keeping the base intact.

3.2 Docker

Containers run directly on the host's kernel, wheter the host is physical or virtual.

The **Docker engine** is a client/server application, where the client is the Command Line Interface (CLI), through which the user can interact, and the server is the Docker daemon.

The **communication** is done through the REST API, that can be also called separately (called from the application).

The server (**daemon**) is the one responsible of creating and managing physically the Docker objects (like networks, containers, images and data volumes).

From the **client** some commands (like docker build, docker pull and docker run) are launched.

The **images** are the templates (read-only) that contains the instructions used to create containers. There is also a registry, where the daemon can download the images.

Containers are simply runnable instances of an image.

An example of a Docker CLI command is `docker run -i -t ubuntu/bin/bash`, which:

- locate and eventually download the ubuntu image (if not locally stored);
- create a new container;
- allocate a read/write file system as final layer on the image;
- isolate the file system created from the host file system;
- create a network interface connected to the default network;
- start the container and run the `/bin/bash` command.

3.2.1 Storage Options

In Docker there are four possible storage options:

- **Container writable layer:** data written here does not persist when the container is terminated. There is reduced portability since this layer is closely related to the host where the container is running. Furthermore the performances are low.
- **Volume:** they are created and managed by Docker and isolated from the host. Data are stored in a specific directory on the Docker host.

The main advantages are:

- volumes are easier to backup or migrate than bind mounts;
 - volumes can be more safely shared among multiple containers;
 - volumes drivers let you store data on a remote host or cloud providers;
 - volumes works both on Windows and Linux;
 - volumes must be managed using Docker CLI commands or the Docker API (you cannot access data directly like a normal file system).
- **Bind mount:** a file or directory on the host machine is mounted into a container. They are shared with host the processes.

The disadvantage is that is not portable since it depends on the host file system.

Performance are usually higher than volumes.

- **Tmpfs:** it's an area of memory outside the container writable layer.
It's a temporary memory that removes data when the container stops.
It's available only in Linux hosts and it's not sharable among containers.

Example

For example:

1. **Volumes** can be used in these situations:
 - (a) to backup, restore or migrate data from a Docker host to an other;
 - (b) to share data among multiple running containers;
 - (c) no guarantee to have a given directory or file structure in the Docker host;
 - (d) to store container's data on a remote host or a cloud provider, rather than locally.
2. **Bind mount** can be used in these situations:
 - (a) to share configuration files from the host machine to the containers;
 - (b) to share source code or to build artifacts between a development environment on the Docker host and a container;
 - (c) the file or directory structure of the Docker host is guaranteed to be consistent with the one the container require.
3. **Tmpfs** can be used in these situations:
 - (a) data should not persist either on the host machine or within the container for security reasons or to manipulate large volumes of non-persistent data with high performance.

3.2.2 Networking

You can connect containers with the following network drivers:

- **Bridge:** it's the default network driver. It's used when applications run on standalone (autonomous) containers that need to communicate;
- **Host:** it's used for standalone containers, in order to remove the network isolation between the containers and the hosts. In this way we allow the container to use the host network;
- **Overlay:** it connects multiple Docker daemons together and enable swarm (cluster) services to communicate with each other.

This strategy removes the need to do the OS routing among them;

- **Macvlan:** it assigns a MAC address to the container in order to make it appear as a real network device. It's the best choice when dealing with legacy applications that expect to be connected to the physical network;
- **None:** no connection for the container.

Example

For example:

1. **Bridges** can be used when multiple containers should communicate on the same Docker host;
2. **Host** network can be used when the network stack should not be isolated from the Docker host;
3. **Overlay** network can be used when containers are running on different Docker hosts and they should communicate;
4. **Macvlan** network can be used when you are migrating from a VM setup or when you need your containers to look like physical hosts on the network.



4. Virtualization

4.1 Virtualization

The virtualization environment consists of a virtualization layer, with an hypervisor or virtual machine monitor (VMM), between the physical hardware (host) and the virtual machine (client)

4.1.1 Main Features

Virtualization guarantees three main characteristics:

- **Increased security:** the VMM controls and filters the activity of the guest, thus preventing some harmful operations from being performed;
- **Managed execution:** the physical resources are converted into virtual resources through some virtualization techniques like sharing, aggregation, emulation and isolation;
- **Portability:** it's possible to transfer and use data and applications from a computing platform to another. A virtual machine is booted from a VM image that is shared in a specific disk format. You can move around VMs keeping or converting this format.

4.1.2 Layered Structure

A computer system is characterized by a layered structure where the levels can communicate with each other through some interfaces.

4.1.3 Privilege of Instructions

The processor instructions can be divided in:

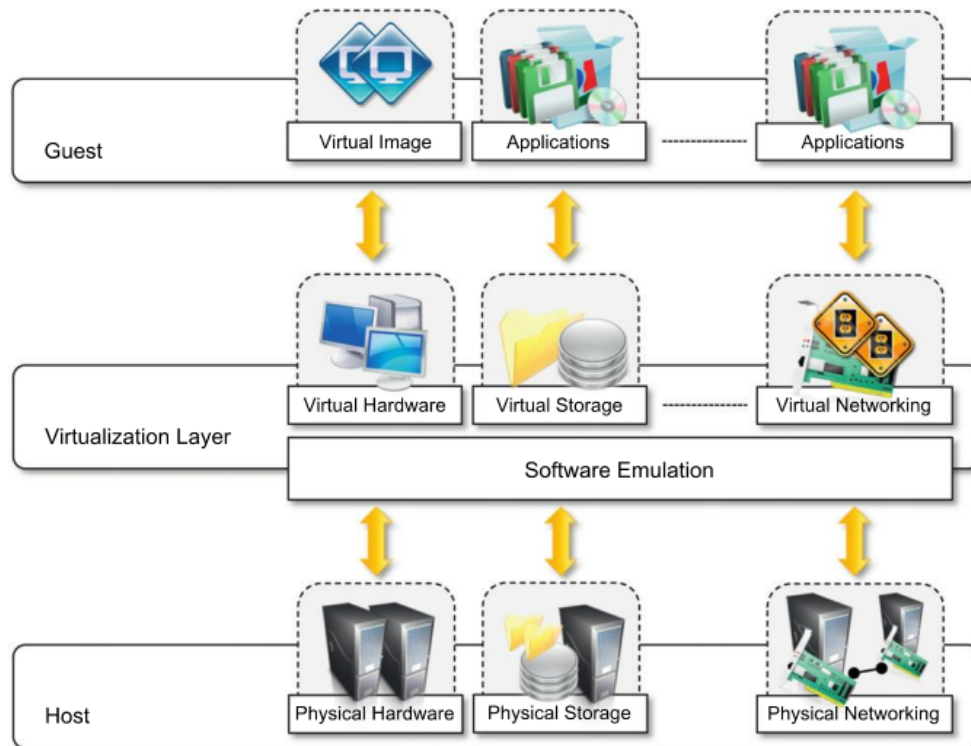


Figure 4.1: Virtualization model

- **Non-privileged:** this one can be used without interfering with other tasks. An example can be arithmetic instructions. This ring of instructions is called User mode;
- **Privileged:** these are executed under specific restrictions and are mostly used for sensitive operations. The VMM runs in supervisor mode (the most privileged instructions).

4.2 Virtual Machine Manager and Hypervisor

4.2.1 Level of Virtualization

There can be two levels of virtualization:

- **Type-II hypervisor:** the hypervisor uses system calls in order to communicate with the operative system;
- **Type-I hypervisor:** the hypervisor uses ISA calls to communicate directly with the hardware.

4.2.2 Properties to Satisfy

In order to create a VMM, three properties must be satisfied:

- **Equivalence:** a guest running under the control of a VMM should exhibit the same behaviour as when it's executed directly on a physical host;

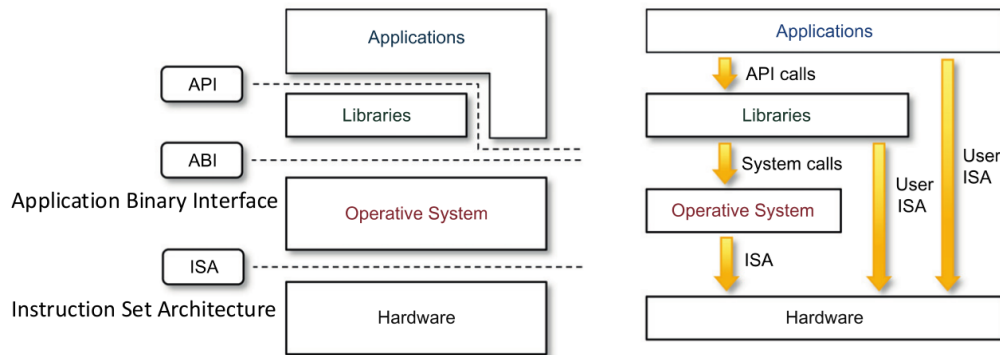


Figure 4.2: Layered structure of a computer system

- **Resource Control:** the VMM should be in complete control of the virtualized resources;
- **Efficiency:** a statistically dominant fraction of the machine instructions should be executed without any intervention of the VMM.

4.2.3 Composition

The VMM consists of three main modules:

- **Dispatcher:** it's the entry point of the virtual machine instance requests that are routed to one of the two other modules;
- **Allocator:** it's responsible for deciding the system resources to be provided to the VM. If the VM instance wants to change the resources allocated to it, the allocator is invoked by the dispatcher;
- **Interpreter:** it's composed of routines that are activated everytime the VM tries to execute a privileged instructions. In this case a trap is executed and the corresponding routine is triggered. In

The Type I hypervisor is installed directly on the hardware. In this case the VMM can be a microkernel on a monolithic architecture.

The trap of the interpreter is triggered when a guest tries to execute an instructions that is reserved to the kernel mode. In this case the hypervisor will control the instruction:

- if the instruction comes from a guest OS then it will execute it safely;
- if the instruction comes from a user program then it will emulate the behaviour of the hardware with such an instruction.

4.3 Full Virtualization

In **Full Virtualization** the OS does not know to be virtualized and only the privileged instructions are intercepted and emulated, while the other ones are passed directly to the hardware, without using the VMM.

4.3.1 Problem of Virtualization in x86 OS

A problem of virtualization was detected on x86 architecture when the sensible instructions don't coincide with the privileged ones. This made difficult to use the trap mechanism.

In order to solve this problem we can use the Binary Translation, which consists of searching for sensible

instructions by analyzing block by block the OS code. When a sensible instruction is found then it will be replaced with a safe call to the hypervisor.

A basic block is a short, straight-line sequence of instructions that ends with a branch (no jum, call, trap, return or any flow alteration).

This dynamic binary translation is not so expensive since most code blocks don't contain sensitive instructions and the other one are cached after be translated such that if the hypervisor will find an equal block later it will not have to translate it newly.

In 2005 Intel and then AMD have introduced specific instructions for virtualization allowing the hardware itself to handle traps natively. However, a lot of trap can paradoxally slow down the performance compared to an excellent binary translation.

4.4 Paravirtualization

In **Paravirtualization** the OS knows to be on a VM and so it directly substitute critical interactions with hypercalls using an intelligent compiler. So hypervisor expose an API that the guest OS uses rather than syscalls. This system (used for example by KVM and Hyper-V) is much more lite and avoids performance losses.

Paravirtualization uses the microkernel architecture. This microkernel handles base elements, like processes, interrupts and scheduling. While in full virtualization librarie communicate with an emulated hardware (causing traps), in paravirtualization we use specific libraries that directly launch hypercalls, extremely simplifying the process.



5. Automation & Orchestration

5.1 Automation vs Autonomic Computing

There is a critical difference between autonomic computing and automation:

- **Autonomic Computing** has the goal of keep the humans out of the loop, letting them only to provide high level objectives;
- **Automation** has the goal of reducing the human interaction and the manual processes

The Automation of IT operations works with software tools and technologies in order to automate organizational and operative processes of IT.

It automates repetitive tasks, orchestrates workflow and integrates automation into existing systems and applications.

The main benefits are:

- **Improved efficiency and productivity:** this is possible because we reduce the time and the effort of manual tasks. This free time of the IT staff can be used to focus on more strategic activities;
- **Reduced human errors:** we eliminate the risk of mistakes caused by manual interventions;
- **Saved time and costs:** we minimize the time wasted for repetitive tasks.

5.1.1 Data Centers Automation

Data centers have specialized platforms to automate key operations like provisioning, configuration, patching and monitoring.

Ansible

For example, Ansible is an open source tool used to manage configuration, deploy and orchestration.

It has two main features:

- **Agentless:** this means that no software is required on the remote machines in order to make them manageable.
This eliminates overhead when the management is not in progress, making this tool ideal for high security and high performant environments.
The instructions are prepared on the central server and then pushed to the hosts.
- **Push Model:** this means that Ansible manages remote machines by using frameworks that already exist on the platform (like SSH or WinRM (Windows Remote Management)).
These framework are used to push the needed code (called modules) on the remote machines.
The remote machines cannot see or affect how the other machines are configured.

The problem is that not all the nodes can run code (like the network devices). In order to automate such systems the module is executed locally on the Ansible control node.

On Linux or Windows nodes the module is copied and executed on the remote node and then deleted.

Ansible Playbooks

Ansible Playbooks use the YAML syntax to allow easy configuration across multiple machines. In this way they are repeatable and reusable. They can:

- declare configurations;
- orchestrate manual processes in a defined order on multiple sets of machines;
- launch tasks synchronously or asynchronously;
- define the system's desired state;
- be idempotent (if applied more than once, the state will not change).

The Playbooks consist of a series of **plays** (each one consists of multiple tasks) that define some automations to apply on a set of hosts (called **inventory**).

A task is a call to an Ansible module that is a small code for doing something that can be simple (like insert a configuration file) or complex (like create entire cloud infrastructures).

The core modules are written in such a way to allow an easy configuration of the desired state. The module checks if the action is actually needed before executing it. For example, if the task is defined to start a webserver then it will be executed only if the webserver is down.

Example

We can consider a three-tier web application. Its environment consists of:

- Applications servers;
- Database servers;
- Content servers;
- Load balancers;
- Monitoring system connected to an alert system (such as a notification service).

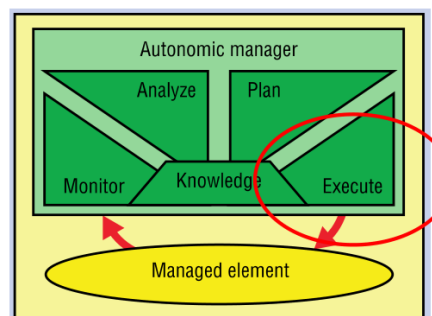
In this scenario Ansible can easily be used to orchestrate a rolling update without interventions. This process would consist of:

1. Consulting a configuration repository to get informations on the involved servers;
2. Configuring the base OS on all the machines and enforcing the desired state;
3. Signaling the monitoring system that the servers will go down for a certain time;
4. Signaling the load balancer to not consider that servers for the load balancing;
5. Stopping the web application servers.
6. Deploying or updating the web application servers code, data and content;
7. Starting the web application servers;
8. Signaling the load balancer that it can newly consider that servers for the load balancing;
9. Signaling the monitoring to resume alerts on that servers;
10. Repeating this process for continuous updatings;
11. Sending email reports and logging as desired.

MAPE

Ansible represents the execute phase in the MAPE paradigm and it has the four self-* properties:

- **Self-Configuration:** when decided the reconfiguration, a playbook can be used to deploy the new system configuration;
- **Self-Optimization:** when decided for adding a new server, a playbook can do all the required actions;
- **Self-Healing:** if a server is degrading, a playbook that detach and reboot the server can be executed;
- **Self-Protection:** if a system is under attack because of vulnerabilities, a playbook for rolling an update can be deployed.



5.2 Orchestration

While automation involves setting up a single task to run autonomously, **Orchestration** involves automating a series of individual tasks so they work together as a single process or workflow.

The orchestration concept could vary depending on the technological context:

- service orchestration;
- IT infrastructure orchestration;
- container orchestration.

5.3 Container Orchestration

The container orchestration allows cloud and application providers to define how to select, deploy, monitor and dynamically control the configuration of multicontainer applications in the cloud.

It handles the deploy, run and maintaining phase, usually offering these main features: resource limit control, scheduling, load balancing, health check, fault-tolerance and auto-scaling.

5.3.1 Kubernetes

Kubernetes (K8s) is a platform for managing containerized workloads and services. This means:

- Service discovery and load balancing;
- Storage orchestration (automatically mounting the storage);
- Automated rollout and rollback (automatically bring the deployed containers to the desired state);
- Automatic bin packing (fit containers onto cluster nodes to make best use of the resources but giving them the needed CPU and memory);
- Self-healing (restart, replace and remove faulty containers);
- Secret and configuration management;

Main Concepts

There are three main concepts in Kubernetes:

- **Object**: they are persistent entities used to represent the state of the cluster. They can describe:
 - which containerized applications are running and on which nodes;
 - the available resources for those applications;
 - the policies (for example restart policies, upgrades and fault-tolerance).

A K8s object is a "record of intent": this means that once the object is created, the system will constantly work to ensure that the object exists.

An object contains two main fields:

- **Spec**: describe the desired state (for example I want 3 instances of the application);
- **Status**: describe the current state of the object (for example there are 2 instances currently up and running);

Example

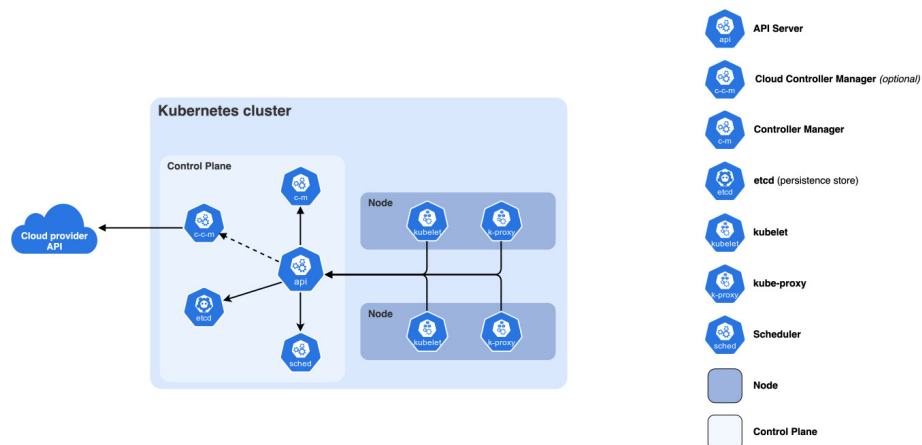
For example, if you create a deployment object with a spec of 3 replicas, the system reads the spec and start the 3 instances and updates the status. If an instance fails (status changes), K8s responds to the difference from the spec and the status by starting a replacing instance.

- **Pod:** it's the smallest deployable unit of computing that can be created and managed in K8s.
A pod models an application-specific "logical host" that contains one or more containers: these containers are tightly coupled, share storage and network resources and the same specification for running themselves.
A pod is similar to a set of containers with shared context (Linux namespace and cgroup) and shared file system.
- **Node:** they are the single virtual or physical machine, member of the K8s cluster or physical.
Each node is managed by the control plane and contains the services necessary to run the pods.
Nodes can enter a cluster through self-registration using the kubelet or through a manual add by the user.

The node status is described by:

- **address:** hostname and IP;
- **conditions:** contains parameters like ready, disk pressure, memory pressure, PID pressure and network unavailable;
- **capacity:** available resources like CPU, memory and maximum number of pods that can be scheduled onto the node;
- **allocable:** amount of resources that are available to be consumed by normal pods;
- **info:** general informations like kernel version, K8s version or OS;

K8s Cluster Components



A Kubernetes (K8s) cluster can be divided into two main components:

- **Control Plane:** is the brain of the system and runs on a dedicated server or on a VM. Its main components are:
 - **kube-apiserver:** is the frontend of the control plane. It scales horizontally;
 - **etcd:** is a key-value database which is highly available for sharing the cluster data;
 - **kube-scheduler:** it's the default scheduler in K8s and selects the optimal node for new pods based on resources requirements, hardware/software constraints and policies. Nodes that meet the scheduling requirements for a pod are called *feasible nodes*. The scheduler finds the feasible nodes (filtering, with PodFitsResources), give them a score (scoring, using some scoring rules), picks the one with the highest score (if there are many with equal score it will choose one at random) and

then notifies the API server about its decision (in a process called binding). If there are no feasible nodes, the pod will remain unscheduled until the scheduler will be able to place it;

- **kube-controller-manager**: runs control processes like the node controller (to detect non active nodes) or job controller (to execute task until completion). It launches control loops that watch the state of a K8s cluster and make or request changes where needed. A controller tracks at least one K8s resource type comparing the spec (desired state) with the current state in order to try to match them via messages to the server API;

Example

To make an example we can consider a job controller. The job is the resource that start one or more pods in order to complete a task and then stop. When the job controller sees a new job, it makes sure that the nodes of the cluster are actually executing the correct number of pods to do that work. The controller, during the task, tells the API server to create or remove pods and update the task state. Once the task is completed, the job controller mark it as *finished*.

Example

We can also consider a controller that monitor external cluster resources, like the Cluster Autoscaler that makes sure that there are enough nodes in the cluster. If not, it is able to horizontally scale the nodes. This controllers read the desired state from the API server, but then communicate directly with an external system to scale the cluster in order to align to the desired state.

- **cloud-control-manager**: links the K8s cluster to the API of a specific cloud provider.
- **Nodes**: they include the following components:
 - **kubelet**: is the agent running on each node in order to control that the containers described in a podspec are running and healthy;
 - **kube-proxy**: is a network proxy running on each node to manage and maintain network rules to allow communication inside and outside the cluster;
 - **container-runtime**: is the software responsible for running containers (for example Docker).

Node Affinity

Node Affinity in K8s is a set of rules used to specify preferences about how pods have to be placed on the nodes. It allows to constrain the pod execution on nodes that have specific labels.

Node affinity is useful for:

- ensuring compliance with data sovereignty laws.

Example

For example, if laws impose that data must be in Europe then node affinity force pods to run only on European nodes.

- optimizing network latency for distributed systems.

Example

For example, if two pods have to communicate a lot then we can force them to run on close nodes.

- allocating resources for high performance computing (HPC).

Example

For example, if an application needs high computation, we can force the pod to run on nodes with GPUs.

- handling workloads with specific storage requirements.

Example

For example if we need a database with quick queries, we can force it to run on nodes with SSD.

- supporting multi-tenancy and resource isolation.

Example

For example we can dedicate specific nodes to a specific enterprise.

Node Anti-Affinity is used to specify rules that prevent the scheduling of pods in the same nodes. It's particularly useful for highly availability (distributing pods across different nodes or zones) and for separating workloads for security or performance reasons.



6. Cloud Storage

6.1 Basic Concepts

6.1.1 Atomicity

An **atomic transaction** is a multi-step operation that should be complete without any interruption.

Atomicity requires non-interruptible ISA operations, like:

- **test-and-set**: writes to a memory location and returns the old content that has been replaced;
- **compare-and-swap**: compares the content of a memory location with a given value and if they are equals, then modifies the content of that memory location with a new value.

Atomicity also need mechanisms to access shared resources and to manage critical section for example with locks, semaphore or monitors.

There are two types of atomicity:

- **All-or-nothing**: it consists of the following phases:
 1. **pre-commit phase**: preparatory actions are done but can be undone.

Example

For example allocation of a resource or fetching a page from a secondary memory;

2. **transition**: is the commit-point;
3. **post-commit phase**: irreversible actions are done. For example write on a file.

In this kind of atomicity, maintaining a history of changes and a log of actions allows to deal with system failures and consistency.

- **Before-or-after:** the result of every read/write is the same both if it occurs completely before or completely after any other read/write.

Example

For example if we have a sequence of read and write operations in a transaction, then the concurrent execution should give the same result if we execute them subsequently.

6.1.2 Storage Model

A storage model is the layout of a data structure in physical storage. It's desirable to have these properties:

- **read/write coherence;**
- **before-or-after atomicity;**
- **all-or nothing atomicity.**

The following are the two storage models:

- **Cell Storage Model:** it's based on the assumption that the memory is organized in cells of the same size and each object fits exactly in one cell.

It reflects the physical organization of common storage media where the primary memory (RAM) is organized as an array of memory cells, while the secondary memory (hard disk) is organized in sectors or blocks.

The read and write operations are done by sectors and blocks .

This storage model guarantees the read/write coherence since the result of a read is the same of the most recent write on the same memory location.

Cells storage guarantees only before-or-after atomicity, but not all-or-nothing atomicity because once the content of a cell is changed by an action, there is no way to abort the action and restore the original content.

- **Journal Storage Model:** it's thought to manage more complex objects, that are saved as records.

It's a system composed by a manager that coordinates the access to the underlying cell storage.

In this model we maintain not only the current value but also the entire history of versions of the variables, which is saved in the log.

The log is provided by an all-or-nothing action that firstly records the action in a log and then installs the changes in the cell storage.

Journal storage guarantees all-or-nothing atomicity thanks to the log that is done before the action (commit). So if one of these operations is aborted we can return to the previous state.

All-or-nothing atomicity implies read/write coherence and before-or-after atomicity.

6.1.3 Consensus

Consensus is the process of agreeing to one of several alternatives proposed by a number of agents (in distributed systems the agents are a set of processes that propose a single value).

An important family of protocols to reach consensus is Paxos, which divided the processes in Proposers, Acceptors and Learners and through a set of operations (prepare, promise, accept, learn) they reach consensus.

6.2 Google File System (GFS)

It's designed to guarantee high reliability despite of frequent hardware and software failures.

GFS files can range in size from few GB to hundreds of TB. They are collected in fixed-size segments called **chunks**. Each chunk is of 64MB, is shared on Linux File System and is replicated on multiple sites (3 by default).

Advantages of large chunks are:

- reduction of metadata maintained by the system;
- reduction of requests to locate the chunks;
- reduction of disk fragmentation.

GFS has a master that controls the chunk servers and maintains metadata such as filenames, access control informations and locations of the replicas.

To recover in case of failures the operation log maintains a historical record of metadata changes. Changes are atomic and become visible only after be recorded on multiple replicas on persistent storage.

To minimize the recovery time, the master periodically checkpoints its state and at recovery time replays only the logs recorder after the last checkpoint.

File access in GFS is handled by the master that grants a lease to a chunk server (primary). The chunk server will be responsible for the updates.

On the other hand, the creation of a file is handled by the master.

The write on a file is done in the following way:

1. Client contacts the master to know which is the primary chunk server and where are the others;
2. Client sends data to all the chunk servers;
3. Chunk servers store data in a LRU (Least Recently Used) buffer and send an ACK to the client;
4. Client sends the write request to the primary;
5. Primary applies the modifications and forward them to the secondaries;
6. Secondaries confirm the operation to the primary;
7. Primary sends an ACK to the client.

6.3 Hadoop Distributed File System (HDFS)

It's a distributed file system written in Java. It's portable but it can't be directly mounted on an existing OS.

It uses blocks of 64-128MB and replicates them on several nodes (3 by default).

This system follows a master/slave architecture with a master (NameNode) which manages the file system, stores metadata, controls access to the files, records changes in a log and checks the DataNodes status.

The slaves are the DataNodes that take care of low level read/write operations.

The write on a file is done in the following way:

1. Client contacts the NameNode to ask for the authorization to write on a file;
2. Client receives the list of the DataNodes for that file;
3. Client contacts directly the primary DataNode;
4. Primary DataNode will contact the secondary and so on;
5. Client starts to send modifications to the primary DataNode;
6. The DataNode applies the modifications and forwards them to the secondary and so on;

7. When each DataNode has applied the modification send an ACK backward on the pipeline, until the client receive all the ACKs.

The read of a file is done in the following way:

1. Client contacts the NameNode to ask for the authorization to read a file;
2. Client receives the list of the DataNodes where each block is stored;
3. Client contacts directly to each DataNode in order to get the blocks;
4. Client will combine the blocks to get the entire file to read.

6.4 Amazon Simple Storage Service (S3)

Data are stored as **objects** into **buckets**. Both objects and buckets are identified by their names.

An user can create a maximum of 100 buckets and he can also set read/write permissions with ACL.

Objects can range from 1 to 5 GB and can be loaded (PUT) or downloaded (GET) entirely or by specific blocks, of bytes.

6.4.1 RW failures

The client application must handle the error in read and write, by retrying until the request succeeds. The MD5 checksum is used to check integrity.

In case of write failures, if a PUT request fails, the client has to retry. The request fails if the MD5 received by S3 (which computes it on the file in the bucket) is different from the one that the client application compute on the local file.

In case of read failures, if the MD5, that is attached to the file that S3 sends to the client, is different from the one that the client computes on the received file, the client can retry the read operation.

6.4.2 Data Consistency Model

S3 offers a strong read-after-write consistency both for PUT and DELETE operations, and for ACL, tag and metadata.

Modifies on a single key are atomic: if a thread execute a PUT and another a GET on the same key, S3 will return either the old or the new data, but never some partially modified data.

A PUT returns a success to the client only when all the replicas have been modified.

If there are two concurrent writers on the same key, the operation with the highest timestamp will win (last-writer-wins). S3 does not support lock mechanism, unless the developer implement it itself.

S3 does not support multi-key operations: so modifies on a key can't affect updates on another key.

The bucket configuration, instead, follows an eventual consistency model: if a bucket is deleted and someone request the list of buckets, the one deleted can still appear for a short time.

6.5 NoSQL

6.5.1 Databases

Databases are an important component in a cloud application. We can have:

- **Relational DBMS:** guarantees atomicity, consistency, isolation and durability (ACID properties).

They are reliable but they have problems to work with too many data that require responses in real time.

To mitigate this problem the memcaching mechanism is used to maintain objects in RAM.

They're also complex to scale in speed and size since it's fundamental to keep the consistency between the several copies of the DB.

- **NoSQL Databases:** lots of modern big data do not require a relational model.

They do not guarantee the ACID properties but the BASE ones (basically available, soft state and eventually consistent).

They are projected to scale horizontally and to eliminate the single point of failure.

6.5.2 BigTable

BigTable is a distributed storage system developed by Google. It was projected in order to be highly applicable, scalable and performant both for mapreduce operation and for latency-sensitive services.

A BigTable is a sparse, distributed, persistent multidimensional sorted map.

It offers a flexible model where the client controls the layout and format of data and their location.

Rows, Columns and Timestamps

Data are treated as sequences of bytes not interpreted and are indexed using row and column names that can be an arbitrary string.

A row key is an arbitrary string of up to 64KB.

Each read and write operation on a single row is atomic, regardless from the number of columns involved.

In a table, rows are ordered lexicographically and are dynamically partitioned into **tablets**, that is the fundamental unit for load balancing.

Column keys are grouped into maximum 100 sets called **column families**, where:

- all the data stored in a column family have the same type;
- column family's data can be compressed;
- families should be rarely changed and the data inside can't be added after the family is created;
- access control and memory accounting is done at family level.

A column key is *family:qualifier*.

Each cell can contain multiple versions of the same data, where each version is indexed by a timestamp (64bit integer).

Timestamp can be assigned by BigTable, that uses the real time in microseconds, or by the client, which has to use a unique identifier.

There are two possible mechanisms for doing garbage collection: only the last n versions are stored or only the versions created in the last n-days are kept.

Architecture

It consists of one master server and many tablet servers. A BigTable cluster stores several tables and each one of them consists of a set of tablets.

The master is responsible for assigning tablets to tablet servers, detecting the addition or expiration of a tablet server, doing load balancing and garbage collection and handling schema changes.

The tablet servers can be dynamically added or removed to accommodate the workload. Each tablet server manages a set of tablets, handles requests for that tablets and splits them if they are too big.

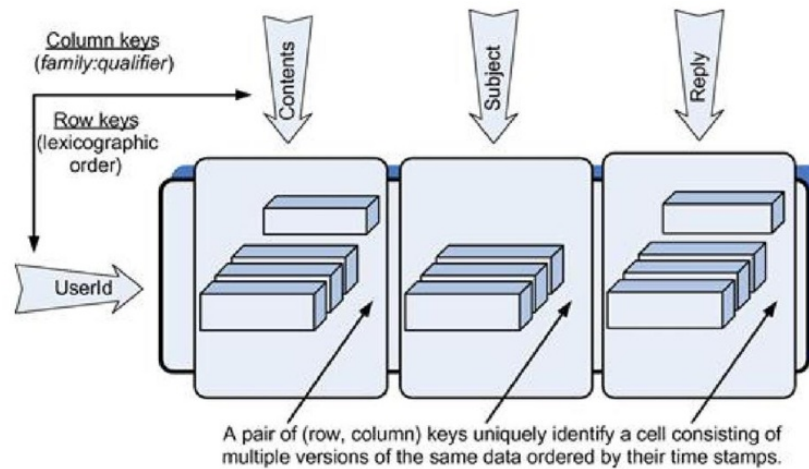


Figure 6.1: BigTable example.

Google SSTable

It's the file format used internally to store Bigtable data. SSTable provides a persistent, ordered immutable key-value map. Each SSTable contains a sequence of blocks, which can be located through a block index and using the binary search.

Chubby

It's a lock service for loosely-coupled distributed system which allows its clients to synchronize their activities and to agree on basic informations about their environment.

Chubby can be used in GFS to elect the master server and to store a small amount of metadata.

Chubby can be used in BigTables to elect the master node, to allow it to discover which tablet server are active and to allow the clients to locate the master and memorize the schema and the ACLs.

6.5.3 Amazon DynamoDB

DynamoDB is the distributed key-value storage of Amazon. It's thought for services that require the access only through primary key, where the relational databases would be inefficient and poor of scaling and availability for those workloads.

Dynamo primary requirement is the high availability and reliability, since the outage would cause significant financial consequences. For this reason Dynamo tradeoffs between availability, consistency, cost-effectiveness and performance.

DynamoDB is eventually consistent: updates spread to all the replicas asynchronously. It exploits optimistic replication techniques: a read/write operation succeeds if a quorum of nodes complete it successfully. This grant that the system is always ready to receive data (always writable), but introduce the risk of conflicting changes that must be detected and resolved in reading phase. These conflicts can be solved both by the data store (with low logic capacity) and by the client application (more complex solution but also more flexible).

Requirements and Principles

Amazon Dynamo has the following design requirements:

- **Query model:** only simple read and write operations based only on primary keys;
- **No ACID:** Dynamo works with lower consistency to improve availability;
- **Efficiency:** Dynamo runs on commercial hardware but following the latency and throughput requirements;
- **Security:** there are not specific security requirements since Dynamo is designed to work only in the Amazon network;
- **Scalability:** it has to scale to hundred of storage nodes.

Amazon Dynamo has the following design principles:

- **Incremental scalability:** it has to be able to scale out one storage host at a time;
- **Symmetry:** every node should have the same set of responsibilities as its peers;
- **Decentralization:** favors decentralized peer-to-peer techniques;
- **Heterogeneity:** ability of balancing the workload across the nodes, also considering their diversity.

System Architecture

- **System Interface:** Dynamo exposes only two API primitives:
 - **Get(key):** locate the replica associate to the key and return the object, or a list of objects if there are version in conflict, with a context;
 - **Put(key, context, object):** determines on which nodes place the object based on the key and then writes the replica on the disk; the context encodes system metadata (such as the object version)

Partitioning Algorithm

It's used to dynamically partition data over a set of nodes in order to grant horizontally scalability. Dynamo distributes data using consistent hashing: each node get a random value which represents its position inside a "ring". In the same way, each data item receive a position in the ring given by the hash of its key.

The problem is that this method can implies a non uniform and unbalanced distribution over the ring. So the virtual node is introduced: a single physical node does not have a unique position in the ring, but it has multiple virtual nodes (the number depends to its computational capacity compared to the others) with different position. If a node becomes unavailable, its workload is divided across the remaining available nodes. When it becomes newly available (or a new node is added), it will receive a workload proportional with the one of the other nodes.

Replication

To achieve high availability and durability, data is replicated on N hosts. The key is firstly assigned to a coordinator node (the first node found on the ring clockwise). The coordinator will save the key locally and will replicate it on the following clockwise N-1 physical nodes.

Data Versioning

Dynamo manage the data versioning in order to achieve the eventual consistency, which allows asynchronously updates.

Since a put() call can return success before all the replicas are actually updated, a following get() call may return a old data. Some applications (like the Amazon chart) prefers this behaviour than reject an "add to

chart".

When there are concurrent changes, there will be divergent versions that Dynamo will treat as immutable objects. The conflicts will be then reconciled.

In order to track causality and history of the changes and to identify which version depends on which other one, Dynamo uses the Vector Clocks, that are pairs (node, counter).



7. FC1 - Definition and Architecture

7.1 Task 1

The goal is to analyze the following two use cases and determine whether they describe cloud services, using the methodology proposed in NIST SP 500-322.

7.1.1 Use Case 1: Video Conference System

: The Cloud Service Provider (CSP) offers a video conference system as a service.

The CSP offers the following features to the Cloud Service Consumer (CSC):

- CSCs can create their accounts.
- CSCs can use a base service for free (basic users) or can subscribe to an enterprise service that offers advanced features and higher performance (enterprise users)
- The service is accessible from the internet, despite the geographical location of the users, by using a standard browser.
- CSCs can start a video conference with one or more peers, who must be subscribers of the service.
- CSCs can record a conference session and store the audio/video files in a storage area managed by the CSP. There is no limit on the number of audio/video files a user can store
- CSCs can set their video conference preferences that are stored by the CSP.
- Despite the number of CSCs connected to a video conference and the number of simultaneous video conferences, the CSCs always perceive the same quality of service experience.

To provide the above features, the CSP operates as follows:

- The CSP uses a front-end website (accessible through the HTTPS protocol) to let users access the

service and all the service features.

- The CSP stores the users' profiles on its storage system. More storage will be added automatically when the number of users increases.
- The CSP stores the users' files in a dedicated storage system. More storage will be added automatically as the number of users and the amount of storage per user increase.
- When a CSC starts a video conference, the service provider automatically allocates network resources to support the communication and computational resources to run the video conference software. The time required to allocate such resources ranges from a few seconds to 1 minute for enterprise users and from tens of seconds to 2 minutes for basic users.
- In case the number of participants in a video conference increases unpredictably, the CSP will allocate more resources automatically, and this will be transparent to the users, with a setup time of a few seconds (1 – 60)
- The video conference software platform used by the CSP for providing video conferencing is the same for all the users, and each time a video conference is started, the CSP creates a new instance of the software platform. The video conference software platform is a complex system composed of many components that can be deployed on independent virtual servers, and some components can be shared among multiple video conferences running simultaneously. This is possible because the software application uses a multi-tenant model. Moreover, physical and virtual resources instantiated to run the video conferencing software instances are shared among the CSCs
- To better manage the infrastructure, guarantee the agreed level of quality of service, and bill the enterprise users according to the pricing policy, the CSP runs a monitoring platform that allows for measuring the usage of the cloud resources, e.g., data traffic, number of computing resources allocated for a video conference, and storage consumed by a user.

7.1.2 Answers Use Case 1

The first use-case is an example of cloud service since it verifies all the essential characteristics:

- **On-demand self-service:** The computing capability can be provisioned without human interaction with the service provider. The Option A is verified because there is fully automated service provisioning since all the operations are automatically done in max. 2 min in a transparent way for the consumers;
- **Broad Network Access:** The computing capability is available from a wide range of locations using standard protocols. The Option A is verified because the services are available over the Internet (The CSP uses a front-end website (accessible through the HTTPS protocol) to let CSCs access the service and all the service features);
- **Resource Pooling:** The computing infrastructure is shared among more than one CSC. Two or more CSCs can share the cloud service resources using a multi-tenant model; in fact the video conference software platform used by the CSP for providing video conferencing is the same for all the users, and each time a video conference is started, the CSP creates a new instance of the software platform. This is possible because the software application uses a multi-tenant model. Moreover, physical and virtual resources instantiated to run the video conferencing software instances are shared among the CSCs;
- **Rapid Elasticity:** The computing capabilities can be “rapidly” provisioned and released to scale. It follows the Option A since resource allocation modification is automated and near-real-time (In case the number of participants in a video conference increases unpredictably, the CSP will allocate more

resources automatically, and this will be transparent to the users, with a setup time of a few seconds (1 – 60));

- **Measured Service:** Cloud service characteristics including resource usage are measured with enough detail to support the requirements of the CSC. To better manage the infrastructure, guarantee the agreed level of quality of service, and bill the enterprise users according to the pricing policy, the CSP runs a monitoring platform that allows for measuring the usage of the cloud resources, e.g., data traffic, number of computing resources allocated for a video conference, and storage consumed by a user.

The cloud computing service model used is SaaS (Software as a Service) since the service that is provisioned is a software application, described as computer programs designed to permit the user to perform a group of coordinated functions, tasks, or activities.

This is an example of public cloud because unrelated CSCs use the shared cloud service and the underlying resources.

7.1.3 Use Case 2: Scientific Computing Platform

The CSP offers to the CSC a scientific computing platform as a service. The platform has the following features:

- The service is accessible from the Internet, despite the geographical location of the CSC, by using a standard browser and standard Internet protocols.
- CSCs can create their accounts.
- CSCs have a web interface to select the resources they want to use. A resource is a Virtual server with scientific software installed. There is a catalog with a broad choice of different combinations of Virtual servers and scientific software. Each combination has a different price per hour of usage.
- When the selection of the resource is completed, the CSC can proceed to the checkout. That means the CSP is informed about the resources to instantiate.
- When a set of resources is instantiated for a CSC, he receives a notification by email with the information needed to access the virtual server via an SSH connection. The CSP guarantees that a CSC request will be served within 36 hours of the checkout time.
- The CSC will pay for the instantiated resources even if not used. If more or different resources are needed, the CSC should go through the selection process described before

To provide the above features, the CSP operates as follows:

- The CSP uses a front-end website (accessible through the HTTPS protocol) to let CSCs access the service and all the service features.
- The CSP stores the CSCs' profiles on its storage system. More storage will be added automatically when the number of CSC' profiles increases.
- The CSP stores the CSC files in a storage volume mounted on the virtual machine instantiated for the CSC. More storage will be added automatically when the number of files increases.
- The CSP operates a back-end office active 365/24/7 where human operators process the CSCs' requests to instantiate virtual servers. The human operators verify that the CSP has sufficient physical servers and storage to accommodate the CSC request, then proceed with automatic instantiation and notification. If no physical resources are available, the request is placed in a queue. The CSP is confident that, within 36 hours, the CSC request can be served.
- The CSCs share the physical machines and storage operated by the CSP. Physical resources are assigned

and reassigned based on CSC demand.

- To bill the CSC according to the pricing policy, the CSP runs a monitoring platform that allows the measurement of the usage of physical and virtual resources.

7.1.4 Answers Use Case 2

The second use-case is not an example of cloud service since it not verifies all the essential characteristics:

- **NO On-demand self-service:** The computing capability are provisioned with human interaction of the service provider. The CSP operates a back-end office active 365/24/7 where human operators process the CSCs' requests to instantiate virtual servers. The human operators verify that the CSP has sufficient physical servers and storage to accommodate the CSC request, then proceed with automatic instantiation and notification. If no physical resources are available, the request is placed in a queue. The CSP is confident that, within 36 hours, the CSC request can be served;
- **Broad Network Access:** The computing capability is available from a wide range of locations using standard protocols. The Option A is verified because the services are available over the Internet (The CSP uses a front-end website (accessible through the HTTPS protocol) to let CSCs access the service and all the service features);
- **Resource Pooling:** The computing infrastructure is shared among more than one CSC. Two or more CSCs can share the cloud service resources using a multi-tenant model; in fact the CSCs share the physical machines and storage operated by the CSP. Physical resources are assigned and reassigned based on CSC demand;
- **NO Rapid Elasticity:** The computing capabilities are not "rapidly" provisioned and released to scale. When a set of resources is instantiated for a CSC, he receives a notification by email with the information needed to access the virtual server via an SSH connection. The CSP guarantees that a CSC request will be served within 36 hours of the checkout time;
- **NO Measured Service:** Cloud service characteristics including resource usage are measured with enough detail to support the requirements of the CSC but the CSC will pay for the instantiated resources even if not used.

The service model used is IaaS (Infrastructure as a Service) since the service that is provisioned is infrastructure. This is an example of public cloud because unrelated CSCs use the shared cloud service and the underlying resources.

7.2 Task 2

7.2.1 Deconstructing the prerequisites

The authors argue that scalability is a "prerequisite" for elasticity. Explain why a system can be scalable without being elastic, and specifically identify the "temporal aspects" that elasticity introduces which scalability lacks.:

A system is scalable if it has the ability of sustaining increasing workloads with adequate performances, adding hardware resources. Scalability is a prerequisite for elasticity, but it does not consider temporal aspects of how fast, how often, and at what granularity scaling actions can be performed. Scalability is the ability of the system to sustain increasing workloads by making use of additional resources, and therefore, in contrast to

elasticity, it is not directly related to how well the actual resource demands are matched by the provisioned resources at any point in time. So a system can be scalable but not elastic.

7.2.2 The Matching Criterion

According to the paper's refined definition, elasticity is measured by how closely available resources match current demand. Analyze the "Matching Function" ($m(w)=r$). Why must this function be derived separately for both upward and downward scaling, rather than assuming a symmetrical relationship?.

The matching function $m(w)=r$ represents the minimum amount of resources r required to satisfy performance requirements for a given workload intensity w . It essentially describes the ideal resource allocation for each workload level. The paper states that this function must be derived separately for upward scaling (scale-out/up) and downward scaling (scale-in/down) because the relationship between workload and resource allocation is not symmetrical in real systems. The system reacts differently depending on whether the workload is increasing or decreasing:

- **Upward scaling:** When workload increases, the system may add resources only after performance thresholds are exceeded;
- **Downward scaling:** When workload decreases, the system usually waits longer before removing resources to avoid oscillations (rapid scaling up and down).

7.2.3 Metrics of Speed vs. Precision

Distinguish between "Speed" and "Precision" as core aspects of elasticity. How would a system that scales "rapidly" but with low "precision" impact both the cloud provider and the end-user?:

Speed: The speed of scaling up is defined as the time it takes to switch from an underprovisioned state to an optimal or overprovisioned state. The speed of scaling down is defined as the time it takes to switch from an overprovisioned state to an optimal or underprovisioned state. The speed of scaling up/down does not correspond directly to the technical resource provisioning/deprovisioning time.

Precision: The precision of scaling is defined as the absolute deviation of the current amount of allocated resources from the actual resource demand. If a system scales "rapidly" (high speed) but with "low precision," it means that the system reacts to workload changes almost instantly, but it consistently fails to calculate and allocate the correct amount of resources. When a low precision system undershoots the actual demand, it fails to provide the minimal amount of resources (r) required to satisfy the system's performance requirements for the current workload intensity (w). The system enters a state of underprovisioning. Because there are fewer resources available than what the workload requires, the system cannot process all incoming requests efficiently. The end-user will experience a severely degraded quality of service, such as slow application response times, timeouts, or complete service unavailability. Conversely, when a low precision system overshoots the actual demand, it allocates far more resources than the workload actually needs. The system enters a state of overprovisioning. The cloud provider ends up powering, dedicating, and maintaining physical infrastructure (such as discrete units of CPU cores or Virtual Machines) that sits completely idle. This results in massive hardware waste and unnecessary financial costs, while also preventing them from leasing those idle resources to other paying customers.

7.2.4 Elasticity vs. Efficiency

Using Figure 3 as a reference, explain why a more efficient system might appear to have "better" elasticity even if its adaptation mechanisms are technically inferior. How do the authors suggest a benchmark should account for this to ensure a "fair comparison"?:

Efficiency expresses the amount of resources consumed for processing a given amount of work. A more

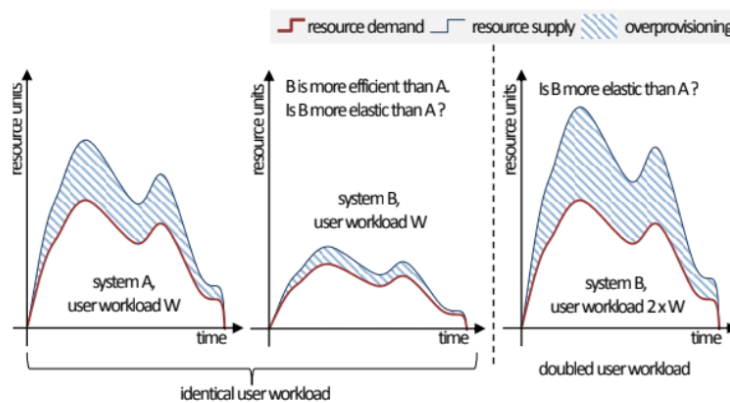


Figure 3: Elasticity vs. Efficiency

efficient system consumes fewer resources for the same work. If we have two systems tested with the same workload, the more efficient one would seem as the more elastic, but only because its adaptation mechanism is not stressed to the same extent as the other system.

To ensure fairness, a benchmark must induce identical demand curves rather than just identical workloads. This requires the benchmark to adjust the workload for the more efficient system (for example doubling the workload for it) so that both the system adaptation mechanisms are stressed at similar intensities.

7.2.5 The Impact of Discrete Scaling Units

The authors note that resources are typically provisioned in discrete units (e.g., VMs, CPU cores). How do these "Resource Scaling Units" prevent a system from ever achieving "perfect" or "optimal" elasticity, even if the adaptation process is instantaneous?:

In real-world scenarios, a system's workload intensity such as incoming user requests or data traffic fluctuates in a smooth, continuous manner. However, cloud systems cannot provision resources in perfectly fluid fractions. As the authors note, resources of a given type can only be provisioned in rigid, discrete units, such as individual CPU cores, Virtual Machines (VMs), or physical nodes. The authors describe optimal elasticity as a hypothetical, perfect scenario where a system has no upper bounds on available resources, and it provisions and de-provisions these resources immediately as they are needed, matching the actual demand exactly at any given point in time. Even if we assume this adaptation process is instantaneous (zero delay in booting up a server or releasing it), a fundamental mathematical mismatch remains. Because demand is continuous but supply must be delivered in blocks (the Resource Scaling Units), the supply curve will always look like a "staircase," while the demand curve looks like a smooth wave. Therefore, even with a technically instantaneous and flawless adaptation mechanism, absolute precision is impossible. There will always be a

residual deviation between the resources allocated and the exact resources needed.

7.2.6 Benchmarking Real-Life Scenarios

The paper suggests that demand curves for benchmarking must include "bursts," "trends," and "seasonal patterns". Select one of these patterns and explain which specific elasticity metric— $\bar{A}$ (average time to scale up) or P_d (average precision of scaling down)—would be most critical for a system to maintain performance during that specific workload behavior.:

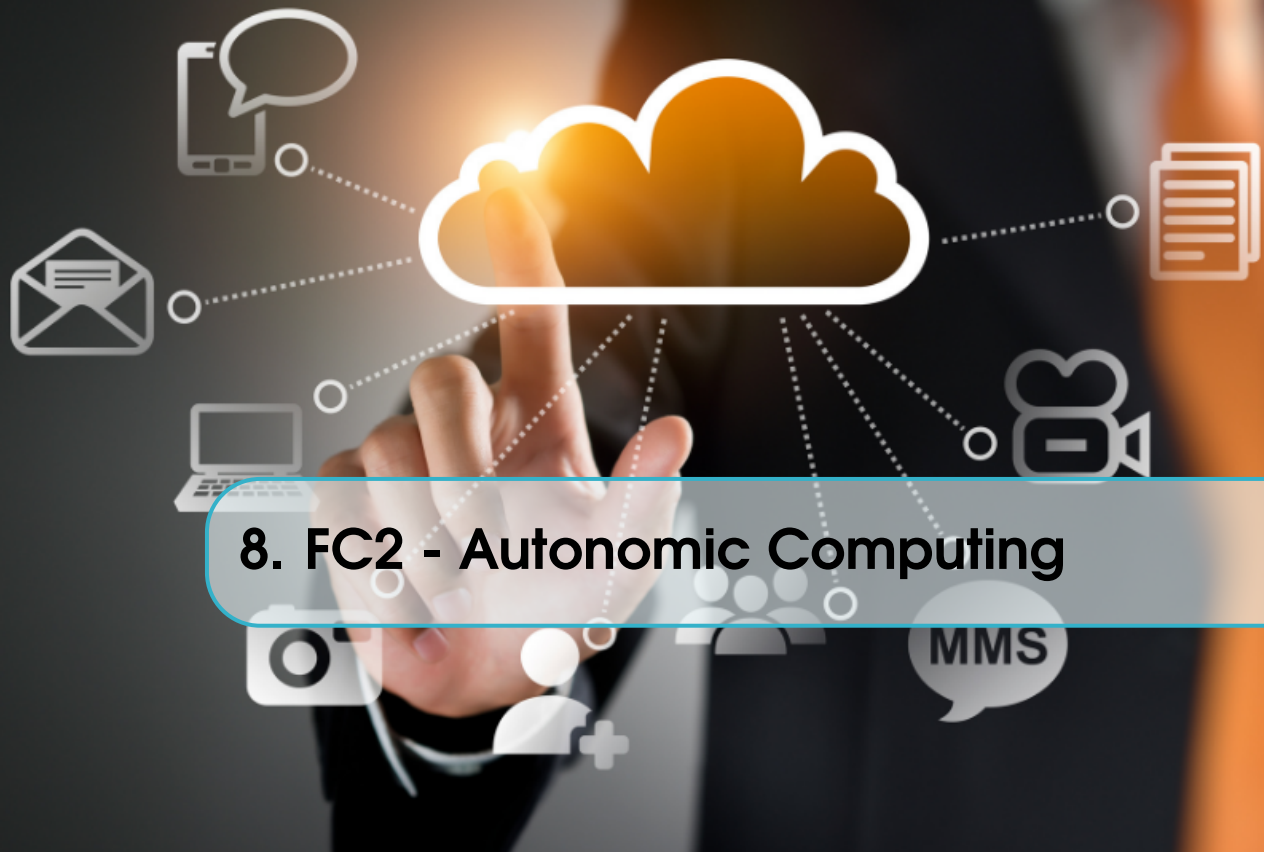
When evaluating a system's ability to handle sudden "bursts" of workload intensity, the most critical metric for maintaining performance is $\bar{A}$ (the average time to scale up).

A burst represents a sudden, sharp increase in workload intensity. This rapid surge immediately pushes the system into an underprovisioned state, meaning the actual demand exceeds the currently provisioned resources.

The paper defines $\bar{A}$ as the average time it takes for a system to switch from an underprovisioned state to an optimal or overprovisioned state. In practical terms, this represents the "average speed of scaling up".

To maintain performance during a burst, the system must provision new resources as fast as possible (minimizing $\bar{A}$) to handle the influx of requests.

Therefore, a fast $\bar{A}$ is essential to ensure the system doesn't buckle under a sudden burst, while P_d is more relevant to the economic efficiency of the system once that burst subsides.



8. FC2 - Autonomic Computing

8.1 Task 1 - Foundations and Inspirations

8.1.1 The Biological Metaphor

Explain how the human Autonomic Nervous System (ANS) serves as the primary inspiration for autonomic computing. Contrast the role of the "conscious brain" in humans with the role of a "human administrator" in a computing system.:

Autonomic Computing takes inspiration from the human Autonomic Nervous System (ANS), which regulates vital functions (like body temperature and oxygen levels) subconsciously to ensure survival. In this metaphor:

- **ANS = Autonomic Manager:** Controls subconscious, low-level activities without wilful human control.
- **Conscious Brain = Human Administrator:** Specifies high-level objectives and only intervenes for major failures or to change system goals.

8.1.2 Homeostasis vs. Vital Parameters

Define homeostasis in the context of both biological and computing systems. What are the "vital parameters" a computing system must maintain within a "survivability zone"?:

Homeostasis in the context of biological systems is a system's ability to maintain internal equilibrium despite changes in its external environment and its internal state. The ANS uses external and internal sensory information to regulate the activity of internal organs so as to maintain a set of vital parameters (body temperature, oxygen levels or glucose concentration) within a 'survivability' zone. In a computing system homeostasis refers to the process of maintaining "vital parameters" within a "survivability zone" without human intervention. These parameters could be functional services and Quality of Service (QoS) properties.

8.1.3 The Role of Motility

According to Section 3.1, how did the biological emergence of motility (movement) necessitate the development of a nervous system? How does this relate to the "change management" required in dynamic software environments?:

While static organisms like plants and fungi were rooted in their environments and did not have a stringent need for a nervous system, the emergence of animal motility brought perpetual environmental shifts. An organism's survival depended on its ability to adapt to new conditions and avoid fatal environments. The nervous system developed as a crucial enabler for this movement, coordinating the actions of various body parts and acting as a "change manager" to help the animal navigate intelligently.

Similarly, software programs were like static organisms; they were isolated entities confined to static environments, carrying out strictly defined calculations. Today, programs are becoming increasingly "agile and mobile," operating collaboratively within highly unpredictable and dynamic environments. Just as the biological nervous system evolved to manage the changes brought on by physical motility, modern software engineering draws inspiration from this biological model. It demonstrates how interconnecting basic system elements into "rapid communication circuits" can allow complex software systems to efficiently and coherently adapt to their ever-changing execution environments.

8.1.4 Scientific Cross-Pollination

Autonomic computing is a consolidation of several fields. Identify three non-computing scientific domains (e.g., from Section 3.1) that contribute models to this field and provide one example of a concept from each.:

Autonomic computing consolidates theories from various domains:

- **Economics:** Virtual currencies for negotiation among processes;
- **Biology:** The ANS metaphor and homeostasis;
- **Control Theory:** Use of feedback loops for regulation.

8.1.5 Microeconomics in Resource Allocation

How are concepts from microeconomics, such as "virtual currencies" and "supply and demand patterns," applied to solve resource allocation problems among competing autonomic processes?:

The concepts of virtual currencies and supply demand patterns are applied through the following mechanisms:

- **Virtual Currencies:** In posted-price negotiation, the provider sets a price in real or artificial currency for its service, and the requester must take it or leave it.
- **Auctions and Brokers:** Autonomic systems will be inhabited by middle agents that serve as intermediaries of various types, including... brokers, auctioneers. These third-party arbiters "can run an auction or otherwise assist these more complex negotiations" over limited resources.
- **Utility Functions:** To manage supply and demand conflicts, Utility functions rely on the definition of a quantitative level of desirability of a given system state. This ensures that when insufficient resources are available, the most desirable combination of available resources can be found.
- **Tendering (Market Dynamics):** To balance dynamic supply and demand, a process called tendering is used to select the node. During this process, the sensor node with the Quality of Context closest to that requested wins the tender, and frequent retendering allows the requester nodes to autonomously adapt to the best source available.

8.1.6 Nash Equilibrium vs. Pareto Optimality

In a multi-agent autonomic system, explain why a Nash Equilibrium (local optimum) might not be Pareto optimal (global optimum). What is required to move a system toward the Pareto boundary?:

In a Nash Equilibrium, every application in the autonomic system makes the best possible decision for itself, assuming no other application changes its behavior. Because applications act selfishly to maximize their own resources, they often settle into a state that harms the overall system's performance. No single app can fix the issue alone, leaving the system stuck in a suboptimal state.

The Pareto Optimality is The Global Optimum: a system reaches the Pareto boundary when it is operating at peak collective efficiency. In this global optimum, it is impossible to give one application more resources without taking them away from another.

Moving from a selfish Nash Equilibrium to a globally efficient Pareto state fundamentally requires cooperation among the agents. Since perfect knowledge of every other agent's strategy is usually impossible in complex systems, the text suggests two workarounds. First, the Stackelberg Leadership model allows one "leader" agent to use its private knowledge to optimize its own responses, which inherently improves the performance of all other agents. Second, Conjectural Equilibrium allows agents to rely on "beliefs" about the system, learned through repeated interactions with their environment, instead of requiring perfect knowledge.

8.1.7 Stackelberg vs. Conjectural Equilibrium

Distinguish between Stackelberg leadership and Conjectural equilibrium. In what scenarios is Conjectural equilibrium preferred due to "imperfect knowledge" of a competitor's strategy?:

Stackelberg Leadership describes a scenario where one specific player acts as a "leader" because they possess private knowledge of all their competitors. By having this broader view, the leader can optimize its responses in a way that coordinates the system. Interestingly, this approach often improves the performance of every player involved, even if the "followers" continue to act myopically without understanding the bigger picture. Conjectural Equilibrium serves as a decentralized alternative for systems where no leader exists and perfect knowledge of others is impossible. In this model, agents replace concrete knowledge with "beliefs" or conjectures about how their competitors will act. These beliefs are not static; they are developed and refined over time through repeated interactions and feedback from the environment.

Conjectural equilibrium is preferred when "imperfect knowledge" prevents agents from knowing the private strategies or internal states of their competitors. This is particularly useful in large-scale, open autonomic systems where it is computationally expensive or security-prohibited for applications to share their internal logic. In these cases, agents cannot calculate a perfect response, so they must instead "learn" to coexist by observing the results of their actions and adjusting their beliefs about the rest of the system accordingly.

8.1.8 Complex Adaptive Systems (CAS)

Define a Complex Adaptive System in the context of Section 3.5. Why is it critical for researchers to distinguish between a system being "extremely complicated" and one having "nonlinear composition"?:

A CAS is a system that is both complex and adaptive, modifying itself in response to environmental changes and learning from the experience. An extremely complicated system is one composed of many parts, while a non linear composition system is one which is not simply a sum of its parts.

8.1.9 Emergence and Self-Organisation

Explain the relationship between self-organization and emergence in CAS. How can simple local interactions between autonomic agents lead to complex global behaviors?:

In a Complex Adaptive System (CAS), self-organization and emergence operate as the "how" and the "what" of decentralised order. Self-organization is the process where a system's internal components interact to create structure or behavior without any external direction or central controller. Emergence is the resulting phenomenon—the complex, high-level patterns or properties that appear at the global level which are not present in, and cannot be predicted by, the individual parts in isolation. Essentially, self-organization is the engine of interaction, and emergence is the surprising collective result it produces.

8.1.10 Ashby's Ultra-Stable System

Describe W. Ross Ashby's double-feedback control system. How does the second loop (meta-management) differ from the first loop (reactive) in terms of the time scale and its impact on "essential variables"?:

Ashby's model is designed to keep "essential variables"—the parameters necessary for survival or stability—within a specific "viability zone.":

1. The First Loop: Reactive Feedback:

- **Function:** This loop acts as a direct response mechanism. It uses a "complicated sensor and actuator system" to oppose external disturbances.
- **Time Scale:** It operates on the short term. It provides immediate, default reactions to common environmental fluctuations.
- **Impact on Essential Variables:** It monitors the variables and triggers a reaction to "bring them back within the physiological zone" as soon as they are pushed toward a limit.

2. The Second Loop: Meta-Management:

- **Function:** This is a higher-level loop that monitors the success of the first loop. If the default reactions fail to stabilize the system (usually due to dramatic environmental changes), this loop triggers an adaptation in the first loop's behavior.
- **Time Scale:** It intervenes over a larger (longer) time scale. It does not react to every small disturbance but rather evaluates the trend of the system over time.

8.2 Task 2 - Autonomic Architectures

8.2.1 The MAPE-K Loop Anatomy

Diagram and explain the four functional parts of the MAPE-K loop: Monitor, Analyze, Plan, and Execute. Why is the Knowledge (K) component considered the "heart" of this architecture?:

Autonomic Manager is composed by 4 part:

1. **Monitor:** This component collects, aggregates and filters details from the managed element and its external environment.
2. **Analyze:** This part provides the mechanisms to observe and understand situations. It correlates the data from the Monitor to determine if the system is meeting its goals or if changes are required to maintain peak performance
3. **Plan:** Based on the analysis, this component constructs the action plans needed to achieve objectives or recover from failures.

4. **Execution:** This component is responsible of the real execution on the managed element. Based on the analysis, this component constructs the action plans needed to achieve objectives or recover from failures.

The **Knowledge (K)** component acts as the central repository, that connect all component together and allows them to work as a single component.

It senses when a variable becomes critical (reaches the absolute viability limit). Instead of directly fixing the variable, it reconfigures the reactive behavior of the first loop so that the system can find a new way to stay within limits.

8.2.2 Touchpoints: Sensors and Effectors

What is the role of touchpoints in an autonomic architecture? How do they allow a manager to apprehend a "managed artefact" that is otherwise too complex to be completely known?:

Touchpoints are the interfaces between the autonomic manager and the managed artefacts. They allow the manager to apprehend a resource's state (via sensors) and influence it (via effectors) without needing to know the resource's internal complexities.

8.2.3 Innate vs. Acquired Knowledge

Distinguish between innate knowledge (engraved in the manager) and knowledge by acquaintance (captured via touchpoints). Provide an example of each in a cloud computing context.:

- **Innate Knowledge (A Priori):** Knowledge "engraved" into the system by designers and architects before the system ever runs.

Example

"If the CPU exceeds 80% of usage, then add a new VM" , the system knows a priori this information.

- **Knowledge by Acquaintance (Acquired):** Information obtained during runtime through direct interaction with the environment and the managed system.

Example

The number of people that are online right now on our system "There are 4200 users online".
The system learns this info by looking at the traffic.

8.2.4 Rule-Based (Reflex) Reasoning

Explain the Event-Condition-Action (ECA) rule mechanism. What are the primary advantages and limitations of using reflex-based reasoning for system adaptation?:

Reflex-oriented managers are typically stateless managers that react to external events in a reflex, non-deliberative manner. In general, such managers are essentially made of event-condition-action (ECA) rules that directly produce adaptation plans (actions) from specific event and condition combinations. Writing adaptation policies is fairly straightforward but can become a tedious task for larger complex systems. Its simplicity and rapidity remains its biggest strength. However, an issue with ECA rules is the problem of

conflicts: an event might satisfy the conditions of two different ECA rules, yet each rule may dictate an action that conflicts with the other. Worse, these conflicts cannot always be detected at the time of writing the policies; some are only detected at runtime. This means that a human may remain in the loop to solve policy conflicts when they arise.

8.2.5 Model-Driven Autonomicity

What is "model-driven autonomicity," and what are the three types of models typically maintained by a manager (structure, environment, non-functional)? How does the separation of concerns apply here?:

Model-driven autonomicity is an approach where an autonomic system uses synchronized models at runtime (model@runtime) to drive its self-management processes. This strategy separates data from logic, allowing the MAPE loop to focus on the "know-how" (decision-making) instead of data collection and representation. The three types of models typically maintained by a manager are:

- **Structure (Architecture) Model:** Represents the software components and their interconnections.
- **Environment Model:** Represents the external context and computing resources where the system operates.
- **Non-Functional Model:** Focuses on specific quality attributes such as security and performance.

Separation of concerns is applied by keeping these models distinct but linked. This allows each model to evolve independently (e.g., updating security rules without changing the architecture), making the manager's code leaner and easier to modify.

8.2.6 Goal-Based vs. Utility-Based Systems

Contrast a goal-based system (which seeks a target state) with a utility-based system (which ranks states). In what scenario would a utility function be necessary to resolve conflicting goals?:

A Goal-Based System classifies all systems strictly as desirable or undesirable. If an ideal state is impossible to reach, the system does not know which among the undesirable states is least bad. Indeed an Utility-based System relies on a quantitative level of desirability of a given system state. They calculate the relative Utility of each combination allowing the system to rank and compare different states. An Utility function is needed to resolve conflicting goals when insufficient resources are available to satisfy all system demands simultaneously. It mathematically allows the autonomic manager to find the best combination of available resources.

8.2.7 Search-Based Reasoning

In Chapter 4, search-based reasoning is described as finding a "path" from a current state to a target state. What are the three components required to define such a search problem?:

To define a search problem within the context of autonomic computing, the text identifies the following three essential components:

1. **Initial and Target States:** The problem begins with a description of the initial state, which represents the current situation of the managed artefacts as captured through system touchpoints. This is paired with a description of the acceptable target states, or goals, which define what the system is trying to achieve.
2. **Set of Actions:** The system must have a defined set of actions that can be performed on the managed artefacts. These actions are executed via the effectors provided by the system and allow the managed resources to be manipulated to effect change.

3. **Transition Model:** A transition model is required to define exactly which actions must be realized to move the system from one state to another. This model essentially maps out the logic of how individual actions result in state changes, allowing the algorithm to navigate the problem space.

8.2.8 Architectural Models and Runtime Reflectivity

How does an Architectural Model reflect the structure and requirements of a managed system? Why is it important that this model is kept "causally connected" to the running software?:

An architectural model represents the managed system as a logical graph where nodes denote components (computational tasks) and arcs denote connectors (communication paths). It reflects system requirements by defining a set of constraints and properties rather than a fixed configuration; for instance, it might mandate a minimum number of active server components to satisfy throughput goals. Additionally, the model mirrors the operating environment by incorporating sensor data such as network latency or user input, allowing the manager to reason about the system's context.

The "causal connection" ensures that the model and the running software remain synchronized, which is fundamental for integrity verification. Because the autonomic manager applies and verifies adaptation plans on the model first, it can guarantee that a change will not violate system constraints before it is executed on the live resources. This connection is also the primary defense against the stale state paradox: without tight synchronization, an autonomic manager might execute a repair plan based on an outdated "belief" about the system state (e.g., adding a server to fix an overload that has already passed), potentially destabilizing the environment.

8.2.9 Probabilistic Reasoning and Uncertainty

Explain the use of Probability of Correctness (PoC) when dealing with multiple Context Providers (CPs). How does combining data from different sensors (like a camera and a pressure pad) reduce uncertainty?:

In dynamic environments, data from sensors is often uncertain or conflicting. Using Probability of Correctness (PoC) allows the system to weigh inputs from different "Context Providers" to reach a reliable conclusion despite "noisy" or incomplete information.

8.2.10 Autonomic Systems That Learn

Describe the most ambitious type of autonomic system—one that includes learning capabilities. How can reinforcement learning be used to update a manager's knowledge or even its utility functions?:

Learning systems use techniques like reinforcement learning (inspired by the biological dopamine reward signal) to improve their performance over time. This allows a generic manager to start with basic "innate" rules and develop sophisticated, specialized behaviors for its specific environment.

8.2.11 Self-management In Practice

Consider the following scenarios and explain which of the self-management capabilities are implemented.:

- **Scenario 1:**

Self-Optimization: the Autonomic Manager constantly analyzes performance metrics (CPU utilization, response time) to balance conflicting objectives: guaranteeing performance thresholds (SLOs) while simultaneously ensuring the cost of virtual resources remains minimal.

- **Scenario 2:**

Self-Healing: the Autonomic Manager achieves this by continuously monitoring the "health state" of the containerized application instances. If it detects that a failure has caused the number of healthy instances to drop below the required threshold of seven, it recovers the system by starting a replacement instance.



9. FC3 - Autoscaling

9.1 Task 1



10. FC4 - Cloud Storage: GFS & HDFS

10.1 Task 1

A person with dark curly hair and glasses is shown from the chest up, wearing a white long-sleeved shirt. They have their mouth wide open in a shout or scream, and their hands are raised near their face, palms facing forward. The background is a solid orange color.

11. FC5 - Cloud DB: Dynamo & BigTable

11.1 Task 1